LAPORAN PRAKTIKUM

PMB



Disusun untuk Memenuhi Tugas

Mata Kuliah:

Workshop DevOps

Oleh:

Ibram Maulana Akhsanul Qasasi

NRP. 3126640014

PROGRAM STUDI D4 LANJUT JENJANG TEKNIK INFORMATIKA

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

2026

PRAKTIKUM UJI COBA PMB

A. Persiapan lingkungan

1) Instalasi Utilitas

```
qasasi@IbramQSS:~$ sudo apt install -y unzip curl python3 python3-yaml openssl make
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
unzip is already the newest version (6.0-28ubuntu4.1).
python3 is already the newest version (3.12.3-0ubuntu2.1).
python3 set to manually installed.
python3-yaml is already the newest version (6.0.1-2build2).
python3-yaml set to manually installed.
Suggested packages:
  make-doc
The following NEW packages will be installed:
  make
The following packages will be upgraded:
  curl libcurl3t64-gnutls libcurl4t64 libssl3t64 openssl
5 upgraded, 1 newly installed, 0 to remove and 194 not upgraded.
Need to get 180 kB/4036 kB of archives.
After this operation, 417 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble/main amd64 make amd64 4.3-4.1build2 [180 kB]
Fetched 180 kB in 2s (106 kB/s)
(Reading database ... 55162 files and directories currently installed.)
Preparing to unpack .../libssl3t64_3.0.13-0ubuntu3.16_amd64.deb ...
Unpacking libssl3t64:amd64 (3.0.13-0ubuntu3.16) over (3.0.13-0ubuntu3.15) ...
Setting up libssl3t64:amd64 (3.0.13-0ubuntu3.16) ...
(Reading database ... 55162 files and directories currently installed.)
Preparing to unpack .../openssl_3.0.13-0ubuntu3.16_amd64.deb ...
Unpacking openssl (3.0.13-0ubuntu3.16) over (3.0.13-0ubuntu3.15) ...
Preparing to unpack .../curl_8.5.0-2ubuntu10.16_amd64.deb ...
Unpacking curl (8.5.0-2ubuntu10.16) over (8.5.0-2ubuntu10.13) ...
Preparing to unpack .../libcurl4t64_8.5.0-2ubuntu10.16_amd64.deb ...
Unpacking libcurl4t64:amd64 (8.5.0-2ubuntu10.16) over (8.5.0-2ubuntu10.13) ...
Preparing to unpack .../libcurl3t64-gnutls_8.5.0-2ubuntu10.16_amd64.deb ...
Unpacking libcurl3t64-gnutls:amd64 (8.5.0-2ubuntu10.16) over (8.5.0-2ubuntu10.13) ...
Selecting previously unselected package make.
Preparing to unpack .../make_4.3-4.1build2_amd64.deb ...
Unpacking make (4.3-4.1build2) ...
Setting up libcurl4t64:amd64 (8.5.0-2ubuntu10.16) ...
Setting up libcurl3t64-gnutls:amd64 (8.5.0-2ubuntu10.16) ...
Setting up make (4.3-4.1build2) ...
Setting up curl (8.5.0-2ubuntu10.16) ...
Setting up openssl (3.0.13-0ubuntu3.16) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.8) ...
```

Gambar 1 Instalasi utilitas praktikum di Ubuntu WSL

Perintah pada Gambar 1 digunakan untuk memasang utilitas ekstraksi ZIP, pengujian HTTP, pemrosesan Python/YAML, pembuatan nilai acak, dan otomasi perintah. Opsi -y memberikan persetujuan otomatis ketika apt meminta konfirmasi. Hasil instalasi berhasil, sehingga kebutuhan awal skrip proyek tersedia.

2) Pemeriksaan direktori awal

```
qasasi@IbramQSS:~$ ls
:USERPROFILE.wslconfig  debian-11.0.0-amd64-netinst.iso  debian-12.10.0-amd64-netinst.iso  devsecops-lab  docker-lab
```

Gambar 2 Daftar direktori awal di Ubuntu

Perintah pada Gambar 2 menampilkan isi direktori saat itu. Hasilnya menunjukkan direktori docker-lab yang digunakan sebagai tempat penyimpanan beberapa praktikum Docker.

3) Pemeriksaan isi docker-lab

```
qasasi@IbramQSS:~$ cd docker-lab
qasasi@IbramQSS:~/docker-lab$ ls
bab-2  bab-3  custom-web
```

Gambar 3 Isi direktori docker-lab sebelum penambahan Bab 6

Perintah pada Gambar 3 digunakan untuk masuk ke direktori kerja dan melihat folder praktikum sebelumnya. Hasil yang ditemukan adalah bab-2, bab-3, dan custom-web, sehingga proyek baru dapat ditempatkan bersama proyek lama tanpa menghapusnya.

4) Menyalin ZIP dari Windows

```
qasasi@IbramQSS:~/docker-lab$ cp "/mnt/c/Users/ibamq/Downloads/PMB_DevSecOps_Lab_Bab06_v1.0.0.zip" .
```

Gambar 4 Penyalinan ZIP PMB Bab 6 ke direktori kerja

Perintah pada Gambar 4 menyalin arsip ZIP dari partisi Windows yang diakses melalui /mnt/c ke direktori docker-lab di WSL. Tanda titik di akhir perintah menunjuk lokasi saat ini. Perintah berhasil tanpa pesan kesalahan.

5) Pemeriksaan ZIP dan ekstraksi



Gambar 5 Arsip PMB berhasil ditemukan dan diekstrak

Perintah pada Gambar 5 memastikan arsip tersedia bersama folder lama, lalu mengekstrak isinya. Output ekstraksi memperlihatkan folder pmb-devsecops-lab beserta file aplikasi, Docker Compose, Fluent Bit, SQL, dokumentasi, dan skrip praktikum.

6) Pemeriksaan struktur proyek

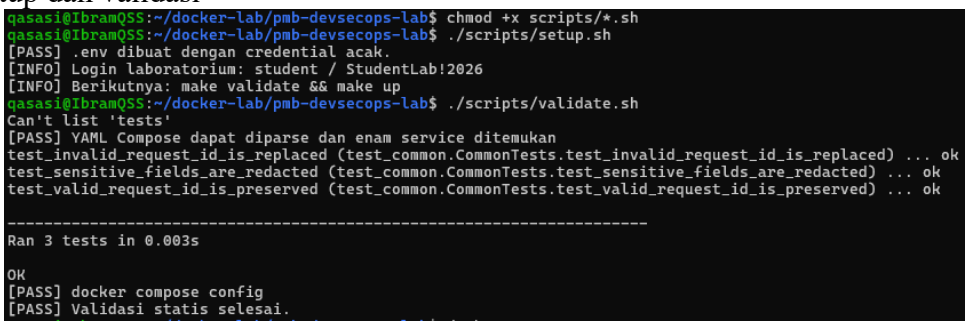
```
qasasi@IbramQSS:~/docker-lab$ ls
PMB_DevSecOps_Lab_Bab06_v1.0.0.zip  bab-2  bab-3  custom-web  pmb-devsecops-lab
qasasi@IbramQSS:~/docker-lab$ cd ~/docker-lab/pmb-devsecops-lab
qasasi@IbramQSS:~/docker-lab/pmb-devsecops-lab$ ls -la
total 76
drwxr-xr-x 9 qasasi qasasi 4096 Oct 27 10:28 .
drwxr-xr-x 6 qasasi qasasi 4096 Oct 27 11:33 ..
-rw-r--r-- 1 qasasi qasasi  740 Oct 27 10:21 .env.example
-rw-r--r-- 1 qasasi qasasi   99 Oct 27 10:21 .gitignore
-rw-r--r-- 1 qasasi qasasi 2916 Oct 27 10:28 MANIFEST.sha256
-rw-r--r-- 1 qasasi qasasi  663 Oct 27 10:21 Makefile
-rw-r--r-- 1 qasasi qasasi 8267 Oct 27 10:26 README.md
-rw-r--r-- 1 qasasi qasasi    6 Oct 27 10:26 VERSION
drwxr-xr-x 5 qasasi qasasi 4096 Oct 27 10:28 app
-rw-r--r-- 1 qasasi qasasi 4649 Oct 27 10:21 compose.yaml
drwxr-xr-x 2 qasasi qasasi 4096 Oct 27 10:28 docs
drwxr-xr-x 2 qasasi qasasi 4096 Oct 27 10:23 fluent-bit
drwxr-xr-x 2 qasasi qasasi 4096 Oct 27 10:23 init-app
drwxr-xr-x 2 qasasi qasasi 4096 Oct 27 10:23 init-log
drwxr-xr-x 2 qasasi qasasi 4096 Oct 27 10:23 queries
drwxr-xr-x 2 qasasi qasasi 4096 Oct 27 10:24 scripts
```

Gambar 6 Struktur direktori pmb-devsecops-lab

Perintah pada Gambar 6 memastikan folder hasil ekstraksi telah terbentuk kemudian menampilkan seluruh berkas proyek, termasuk berkas tersembunyi. Opsi -l menampilkan informasi rinci, sedangkan -a menyertakan berkas seperti .env.example dan .gitignore. File penting yang terlihat antara lain

compose.yaml, README.md, app, docs, fluent-bit, init-app, init-log, queries, dan scripts.

7) Setup dan validasi

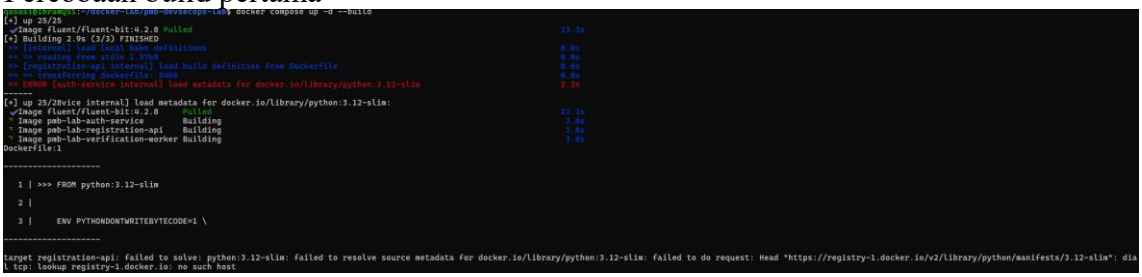


Gambar 7 Persiapan .env dan hasil validasi proyek

Perintah pada Gambar 7 memberikan izin eksekusi untuk skrip shell, menyiapkan konfigurasi .env dan memvalidasi proyek. Hasil setup menyatakan kredensial acak berhasil dibuat. Validasi memastikan enam service dikenali, tiga unit test lulus, dan Docker Compose valid. Pesan “Can't list 'tests'” tidak menggagalkan pemeriksaan karena unit test pada app/tests tetap dijalankan dengan hasil OK.

B. Build container dan pemecahan gangguan jaringan

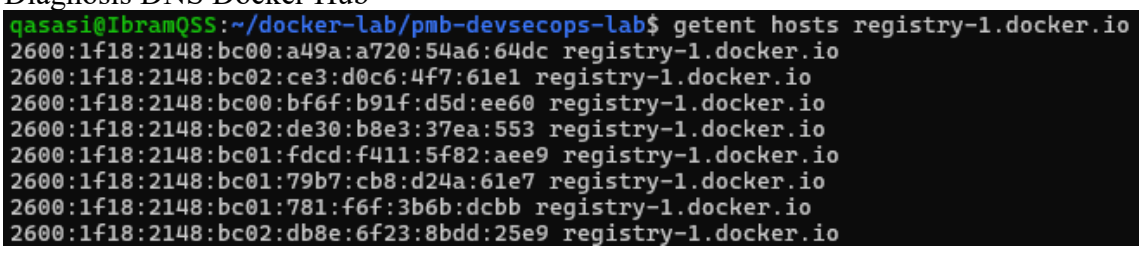
1) Percobaan build pertama



Gambar 8 Build pertama gagal mengambil base image Python

Perintah pada Gambar 8 digunakan untuk membangun image aplikasi dan menjalankan service berdasarkan Docker Compose. Opsi -d menjalankan service di latar belakang, sedangkan --build meminta pembangunan image aplikasi. Proses pertama gagal karena Docker tidak dapat melakukan lookup registry-1.docker.io ketika mengambil metadata python:3.12-slim.

2) Diagnosis DNS Docker Hub



Gambar 9 Hasil resolusi DNS registry Docker Hub

Perintah pada Gambar 9 memeriksa apakah alamat Docker Hub bisa diterjemahkan menjadi alamat IP. Output menampilkan beberapa alamat IPv6, yang menunjukkan bahwa resolusi DNS dari Ubuntu WSL saat diperiksa sudah

berhasil; gangguan saat build kemungkinan bersifat sementara atau terjadi pada sisi Docker Engine.

3) Pengujian endpoint aplikasi sebelum berhasil berjalan

```
qasasi@IbramQSS:~/docker-lab/pmb-devsecops-lab$ curl http://localhost:8080/healthz
curl: (7) Failed to connect to localhost port 8080 after 7 ms: Couldn't connect to server
```

Gambar 10 Endpoint PMB belum dapat diakses

Perintah pada Gambar 10 mencoba mengakses endpoint kesehatan registration-api. Hasilnya gagal karena aplikasi belum berhasil berjalan dan/atau port host belum dapat diakses pada saat pengujian.

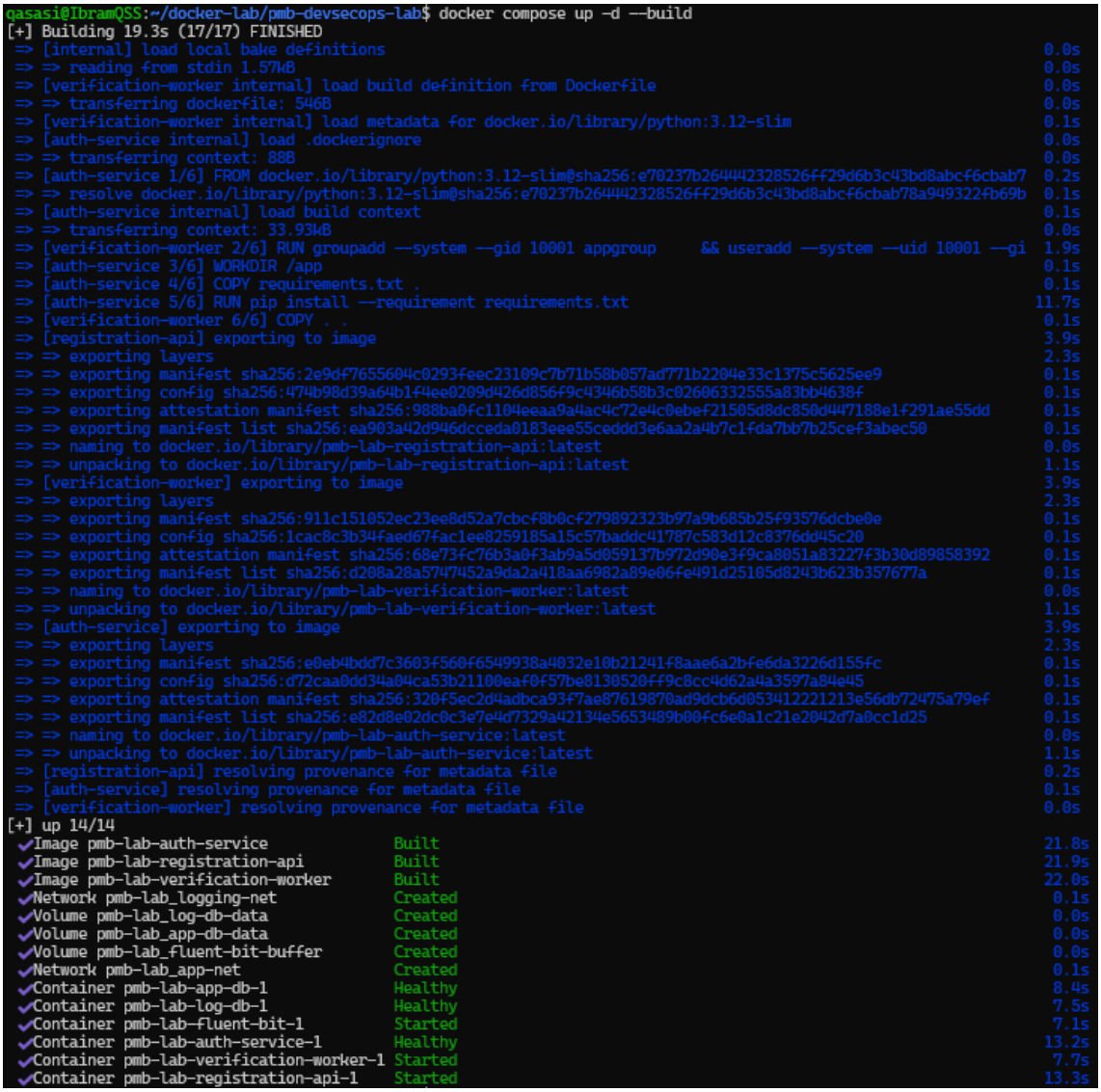
4) Pengunduhan base image secara manual

```
qasasi@IbramQSS:~/docker-lab/pmb-devsecops-lab$ docker pull python:3.12-slim
3.12-slim: Pulling from library/python
86586d55cc56: Pull complete
f5b253875fcb: Pull complete
f2be67225471: Pull complete
ca30a39e3f58: Pull complete
4fd8f7105b68: Download complete
37b2e18f1119: Download complete
Digest: sha256:e70237b264442328526ff29d6b3c43bd8abcf6cbab78a949322fb69b7d91d6b6
Status: Downloaded newer image for python:3.12-slim
docker.io/library/python:3.12-slim
```

Gambar 11 Image Python 3.12-slim berhasil diunduh

Perintah pada Gambar 11 mengambil image dasar Python yang diperlukan oleh Dockerfile aplikasi. Hasilnya menunjukkan seluruh layer berhasil diunduh dan image python:3.12-slim tersedia secara lokal.

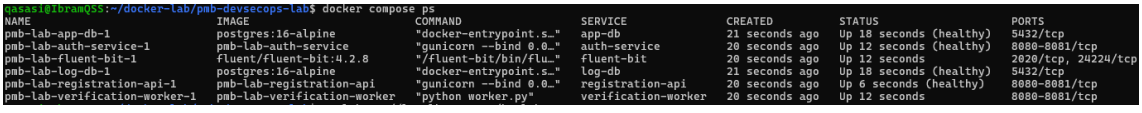
5) Build ulang dan pembuatan enam container



Gambar 12 Build ulang aplikasi PMB berhasil

Perintah pada Gambar 12 membangun ulang ketiga image aplikasi Python dan menyiapkan jaringan, volume, serta enam container. Hasilnya menunjukkan image pmb-lab-registration-api, pmb-lab-auth-service, dan pmb-lab-verification-worker berhasil dibangun dan service mulai dijalankan.

6) Pemeriksaan container



Gambar 13 Status enam service setelah proses build

Perintah pada Gambar 13 menunjukkan container app-db, auth-service, fluent-bit, log-db, registration-api, dan verification-worker. Database dan service yang memiliki healthcheck tampil healthy, tetapi pada saat ini kolom PORTS belum menunjukkan publikasi port registration-api dan Fluent Bit ke host.

7) Uji akses port sebelum perbaikan

```
qasasi@IbramQSS:~/docker-lab/pmb-devsecops-lab$ curl -i http://127.0.0.1:8080/healthz
curl: (7) Failed to connect to 127.0.0.1 port 8080 after 1 ms: Couldn't connect to server
qasasi@IbramQSS:~/docker-lab/pmb-devsecops-lab$ curl -i http://127.0.0.1:2020/api/v1/health
curl: (7) Failed to connect to 127.0.0.1 port 2020 after 0 ms: Couldn't connect to server
```

Gambar 14 Koneksi ke registration-api dan Fluent Bit masih ditolak

Perintah pada Gambar 14 mengirim permintaan HTTP ke endpoint kesehatan aplikasi dan collector serta meminta header respons ditampilkan melalui opsi -i. Kedua permintaan gagal tersambung, walaupun container berstatus berjalan, sehingga perlu memeriksa pemetaan port.

8) Pemeriksaan pemetaan port container

```
qasasi@IbramQSS:~/docker-lab/pmb-devsecops-lab$ docker port pmb-lab-registration-api-1
qasasi@IbramQSS:~/docker-lab/pmb-devsecops-lab$ docker port pmb-lab-fluent-bit-1
```

Gambar 15 Tidak ditemukan pemetaan port yang dipublikasikan

Perintah pada Gambar 15 memeriksa pemetaan port yang diterapkan pada dua container. Tidak ada output yang ditampilkan, menandakan tidak ada port host yang dipublikasikan pada saat itu. Kondisi tersebut menjelaskan kegagalan akses melalui curl.

C. Perbaikan akses jaringan dan website PMB

1) Membuat Compose override

```
qasasi@IbramQSS:~/docker-lab/pmb-devsecops-lab$ cat > compose.override.yaml <<'EOF'
services:
  registration-api:
    networks:
      - app-net
      - host-access
    ports:
      - "127.0.0.1:18080:8080"

  fluent-bit:
    networks:
      - logging-net
      - host-access

networks:
  host-access:
    driver: bridge
EOF
```

Gambar 16 Membuat dan mengisi file compose.override.yaml

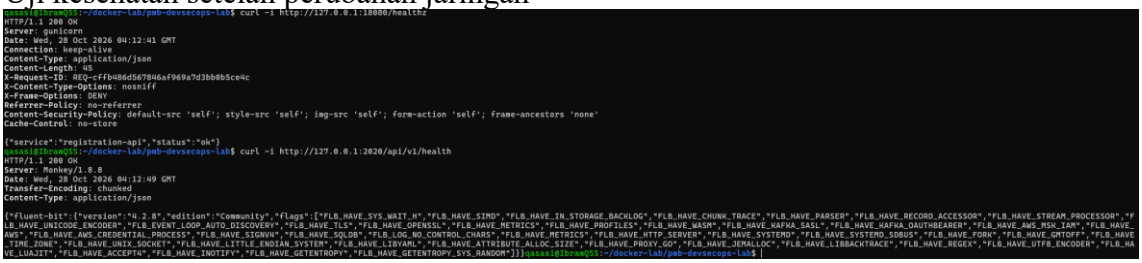
Perintah tersebut membuat file compose.override.yaml tanpa mengubah compose.yaml bawaan dosen. Jaringan host-access ditambahkan pada registration-api dan Fluent Bit, sedangkan jaringan internal utama tetap dipakai. Perubahan ini ditujukan untuk memungkinkan publikasi port host pada lingkungan Docker WSL yang digunakan.

2) Validasi konfigurasi dan penerapan override

```
qasasi@IbramQSS:~/docker-lab/pmb-devsecops-lab$ docker compose config --quiet
qasasi@IbramQSS:~/docker-lab/pmb-devsecops-lab$ docker compose up -d
[+] up 6/6
 ✔Container pmb-lab-auth-service-1         Healthy                                                
```

Perintah konfigurasi memeriksa berkas Compose gabungan tanpa menampilkan output apabila valid; opsi --quiet adalah penulisan yang benar. Setelah perubahan diterapkan, kolom PORTS menunjukkan 127.0.0.1:8080->8080/tcp untuk registration-api serta port 2020 dan 24224 untuk Fluent Bit. Status ini dapat ditampilkan ulang dengan docker compose ps.

3) Uji kesehatan setelah perubahan jaringan



Gambar 18 Kedua endpoint mengembalikan HTTP 200 OK

Perintah di atas menguji ulang kedua endpoint. Hasil registration-api berupa HTTP 200 OK dengan JSON {"service":"registration-api","status":"ok"}, sedangkan Fluent Bit mengembalikan HTTP 200 OK beserta identitas versinya 4.2.8. Hal ini membuktikan akses host ke dua service sudah berfungsi.

4) Konflik port dengan Apache Windows



Gambar 19 Tampilan pada browser ketika mengakses localhost:8080

Ketika alamat http://localhost:8080 dibuka melalui browser Windows, halaman yang tampil justru Welcome to XAMPP.

```
PS C:\WINDOWS\system32> curl.exe http://127.0.0.1:8080/healthz
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
<hr>
<address>Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.2.12 Server at 127.0.0.1 Port 8080</address>
</body></html>
PS C:\WINDOWS\system32> Get-NetTCPConnection -LocalPort 8080 -State Listen

LocalAddress                        LocalPort RemoteAddress                       RemotePort State       AppliedSetting
------------                        --------- -------------                       ---------- -----       --------------
::                                  8080      ::                                  0          Listen
0.0.0.0                             8080      0.0.0.0                             0          Listen
```

Gambar 20 Pemeriksaan port 8080 pada powershell

Pemeriksaan PowerShell memperlihatkan respons Apache/2.4.58 (Win64) dan proses PID 4800 yang mendengarkan port 8080. Dengan demikian, port 8080 pada Windows telah digunakan aplikasi lain dan tidak menunjuk website PMB.

5) Menambahkan port alternatif

```
  GNU nano 7.2
services:
  registration-api:
    networks:
      - app-net
      - host-access
    ports:
      - "127.0.0.1:18080:8080"

  fluent-bit:
    networks:
      - logging-net
      - host-access

networks:
  host-access:
    driver: bridge
```

Gambar 21 Menambahkan port

Pengaturan port 18080 pada host diarahkan ke port 8080 di dalam container. Pengubahan dilakukan pada file override agar konfigurasi utama dari ZIP tetap dipertahankan. Port 18080 dipilih untuk menghindari benturan dengan Apache Windows.

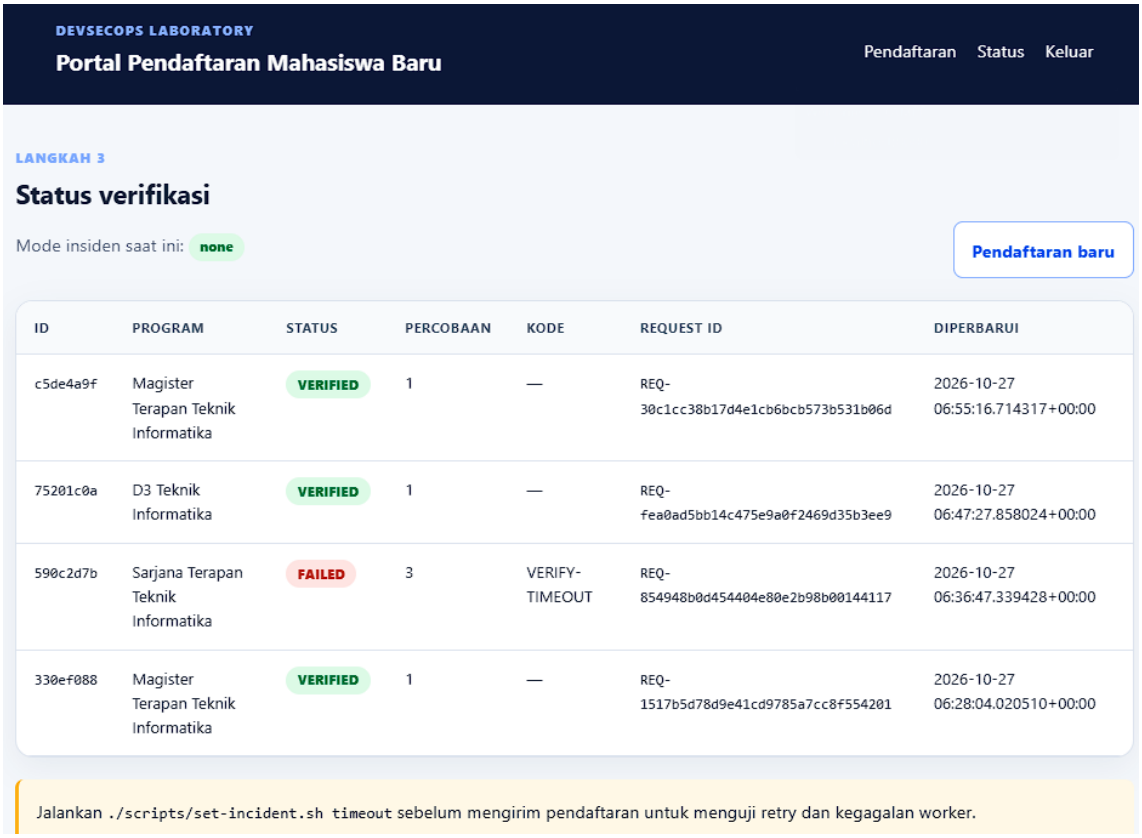
6) Penerapan port alternatif

```
qasasi@IbramQSS:~/docker-lab/pmb-devsecops-lab$ docker compose config --quiet
qasasi@IbramQSS:~/docker-lab/pmb-devsecops-lab$ docker compose up -d
[+] up 6/6
 ✔Container pmb-lab-auth-service-1         Healthy                                                                                                    1.2s
 ✔Container pmb-lab-registration-api-1     Running                                                                                                    0.0s
 ✔Container pmb-lab-verification-worker-1  Running                                                                                                    0.0s
 ✔Container pmb-lab-app-db-1               Healthy                                                                                                    1.2s
 ✔Container pmb-lab-log-db-1               Healthy                                                                                                    0.7s
 ✔Container pmb-lab-fluent-bit-1           Running                                                                                                    0.0s
qasasi@IbramQSS:~/docker-lab/pmb-devsecops-lab$ docker compose ps
NAME                          IMAGE                        COMMAND                  SERVICE              CREATED       STATUS                   PORTS
pmb-lab-app-db-1              postgres:16-alpine           "docker-entrypoint.s…"   app-db               23 hours ago  Up 5 hours (healthy)     5432/tcp
pmb-lab-auth-service-1        pmb-lab-auth-service         "gunicorn --bind 0.0…"   auth-service         23 hours ago  Up 5 hours (healthy)     8080-8081/tcp
pmb-lab-fluent-bit-1          fluent/fluent-bit:4.2.8      "/fluent-bit/bin/flu…"   fluent-bit           23 hours ago  Up 5 hours               127.0.0.1:2020->2020/tcp, 127.0.0.1:24224->24224/tcp
pmb-lab-log-db-1              postgres:16-alpine           "docker-entrypoint.s…"   log-db               23 hours ago  Up 5 hours (healthy)     5432/tcp
pmb-lab-registration-api-1    pmb-lab-registration-api     "gunicorn --bind 0.0…"   registration-api     23 hours ago  Up 5 hours (healthy)     127.0.0.1:8080->8080/tcp, 8081/tcp, 127.0.0.1:18080->8080/tcp
pmb-lab-verification-worker-1 pmb-lab-verification-worker  "python worker.py"       verification-worker  23 hours ago  Up 5 hours               8080-8081/tcp
```

Gambar 22 Menjalankan ulang command penerapan config

Konfigurasi baru dinyatakan valid dan service diterapkan ulang. Hasil status container menunjukkan pemetaan tambahan 127.0.0.1:18080->8080/tcp pada registration-api sehingga website dapat diakses tanpa menghentikan Apache Windows.

7) Membuka portal PMB



Gambar 23 Website PMB berhasil dibuka dari browser Windows

Setelah menggunakan port 18080, browser berhasil menampilkan portal Penerimaan Mahasiswa Baru. Website dapat digunakan untuk login, mengisi formulir, dan memeriksa status verifikasi. Alamat ini merupakan akses lokal praktikum, bukan website produksi.

D. Tahap A — Baseline

1) Mengaktifkan mode normal

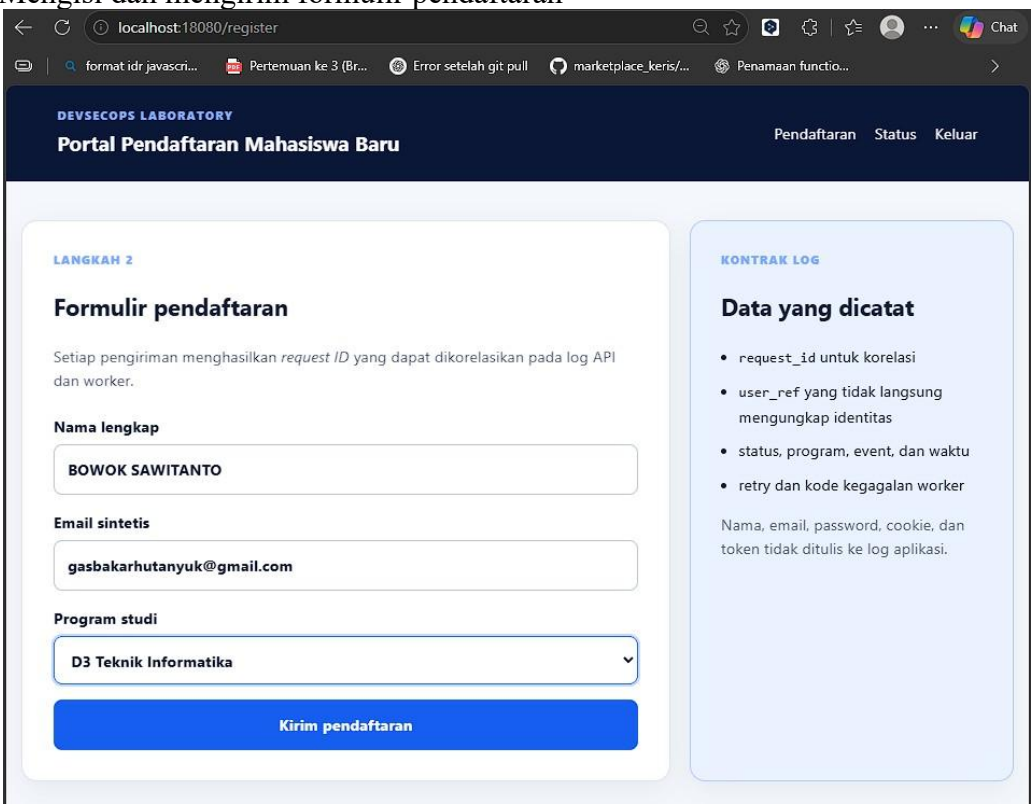
```
qasasi@IbramQSS:~/docker-lab/pmb-devsecops-lab$ ./scripts/set-incident.sh none
UPDATE 1
      key       | value |          updated_at
----------------+-------+-------------------------------
 incident_mode | none  | 2026-10-28 06:52:57.861979+00
(1 row)

[PASS] Mode insiden: none
```

Gambar 24 Mengaktifkan mode none

Perintah pada Gambar 24 menjalankan skrip set-incident.sh dengan parameter none untuk menonaktifkan simulasi gangguan. Output UPDATE 1 dan [PASS] Mode insiden: none menunjukkan mode berhasil diterapkan sebelum pendaftaran normal dilakukan.

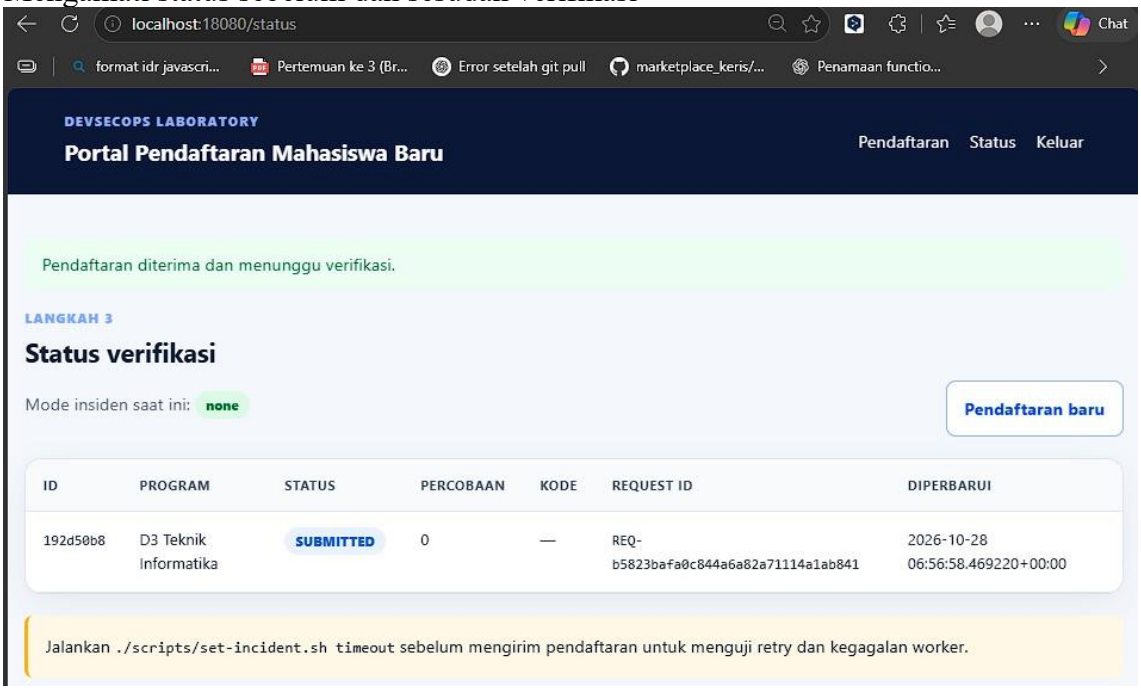
2) Mengisi dan mengirim formulir pendaftaran



Gambar 25 Mengisi formulir pendaftaran Baseline

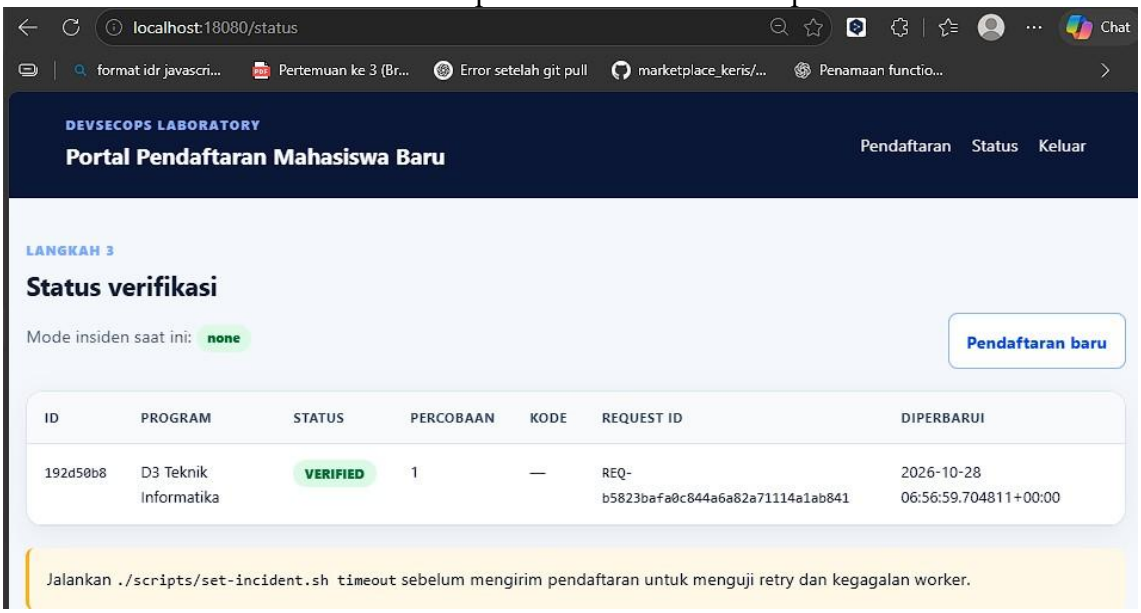
Gambar 25 menunjukkan pengisian formulir PMB melalui localhost:18080/register. Data pendaftaran dikirim untuk memperoleh Request ID yang nantinya digunakan sebagai kunci korelasi antara database transaksi dan centralized log.

3) Mengamati status sebelum dan sesudah verifikasi



Gambar 26 Status SUBMITTED sebelum pembaruan halaman

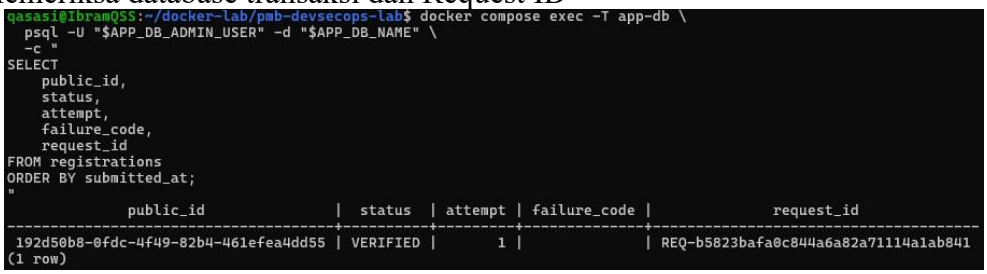
Gambar 26 memperlihatkan status awal SUBMITTED dengan percobaan 0. Kondisi tersebut menunjukkan transaksi telah diterima aplikasi, tetapi proses verifikasi oleh worker belum selesai pada saat halaman ditampilkan.



Gambar 27 Status VERIFIED sesudah pembaruan halaman

Gambar 27 menunjukkan status transaksi berubah menjadi VERIFIED pada percobaan pertama tanpa kode kegagalan. Dengan demikian proses pendaftaran normal dapat diselesaikan oleh verification-worker.

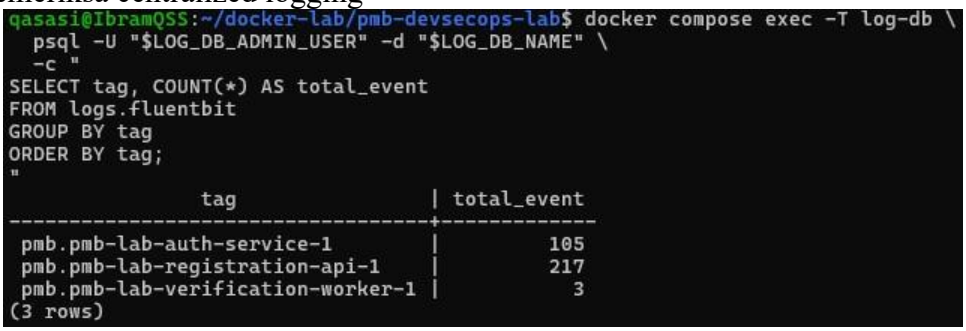
4) Memeriksa database transaksi dan Request ID



Gambar 28 Pemeriksaan hasil transaksi pada app-db

Perintah pada Gambar 28 mengambil public_id, status, attempt, failure_code, dan request_id dari tabel registrations. Hasil query memuat satu transaksi VERIFIED dengan attempt 1 dan Request ID REQ-b5823bafa0c844a6a82a71114a1ab841, yang dipakai untuk menelusuri log berikutnya.

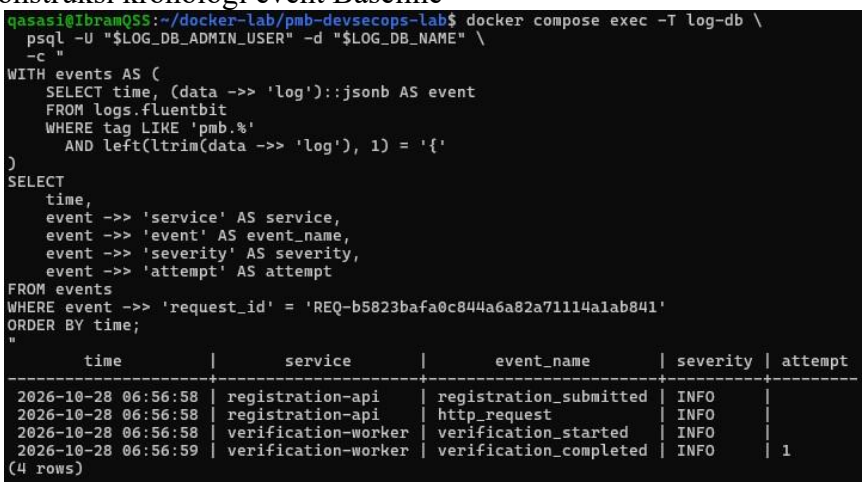
5) Memeriksa centralized logging



Gambar 29 Jumlah log yang tersimpan menurut service

Perintah pada Gambar 29 mengelompokkan record pada logs.fluentbit berdasarkan tag. Hasilnya mencatat 105 log auth-service, 217 log registration-api, dan 3 log verification-worker; angka total 325 mencakup log lain seperti healthcheck, bukan hanya transaksi pengujian.

6) Merekonstruksi kronologi event Baseline



Gambar 30 Kronologi event Baseline berdasarkan Request ID

Perintah pada Gambar 30 menelusuri log terstruktur dengan Request ID Baseline. Ditemukan empat event berurutan: registration_submitted, http_request, verification_started, dan verification_completed; seluruhnya berlevel INFO. Timestamp tercatat pada rentang 06:56:58–06:56:59 UTC, sementara waktu pembaruan status menunjukkan selang sekitar 1,24 detik.

E. Tahap B — Failure Injection

1) Mengaktifkan simulasi timeout

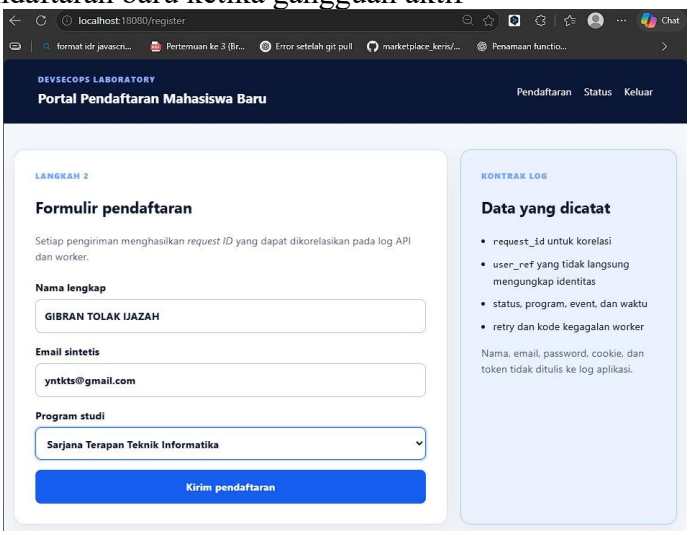
```
qasasi@IbramQSS:~/docker-lab/pmb-devsecops-lab$ ./scripts/set-incident.sh timeout
UPDATE 1
      key       |  value  |          updated_at
----------------+---------+-------------------------------
 incident_mode | timeout | 2026-10-28 07:23:25.778044+00
(1 row)

[PASS] Mode insiden: timeout
```

Gambar 31 Mengubah mode insiden menjadi timeout

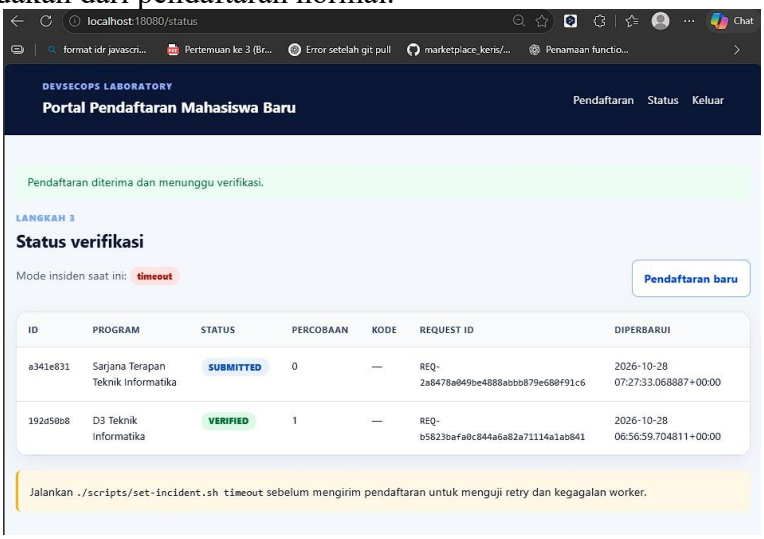
Perintah pada Gambar 31 menjalankan set-incident.sh timeout. Hasil UPDATE 1 dan [PASS] Mode insiden: timeout membuktikan pengaturan gangguan aktif sehingga proses verifikasi berikutnya akan mengalami timeout yang disimulasikan.

2) Mengirim pendaftaran baru ketika gangguan aktif



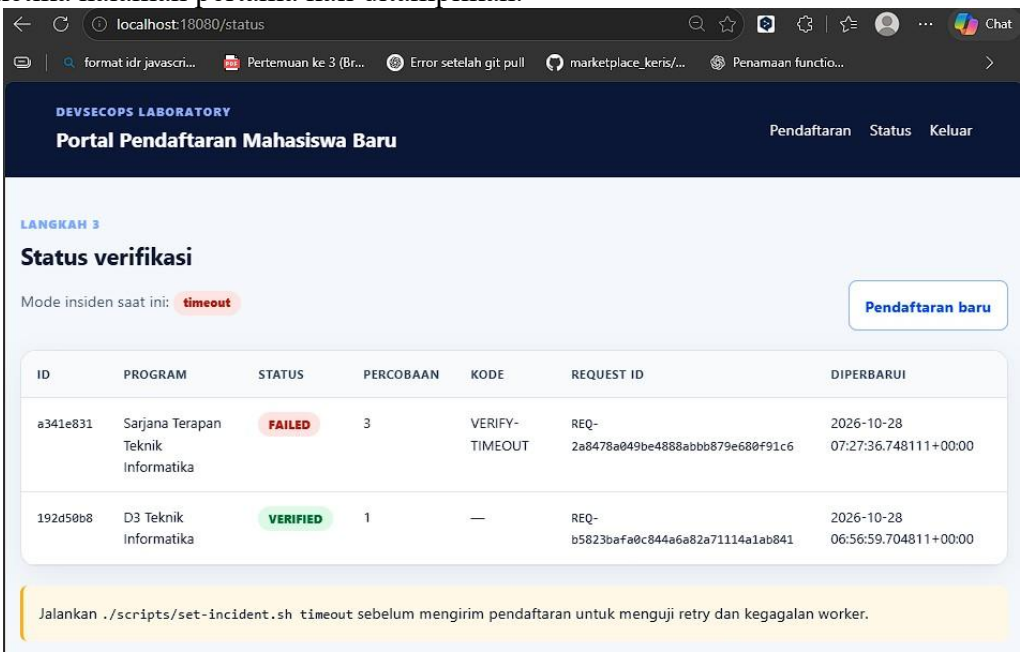
Gambar 32 Formulir pendaftaran baru pada mode timeout

Gambar 32 menunjukkan formulir pendaftaran baru dengan data berbeda dari Tahap A. Pengujian memakai transaksi terpisah agar kronologi dan dampaknya dapat dibedakan dari pendaftaran normal.



Gambar 33 Status awal transaksi Failure Injection

Gambar 33 menunjukkan transaksi baru pada mode timeout masih berstatus SUBMITTED dan memiliki percobaan 0, sehingga kegagalan belum ditetapkan ketika halaman pertama kali ditampilkan.



Gambar 34 Status akhir FAILED setelah tiga percobaan

Gambar 34 memperlihatkan status akhir FAILED dengan jumlah percobaan 3 dan kode VERIFY-TIMEOUT. Sementara itu, transaksi Baseline tetap berada pada status VERIFIED.

3) Membandingkan status melalui database transaksi

```
qasasi@IbramQ55:~/docker-lab/pmb-devsecops-lab$ docker compose exec -T app-db \
  psql -U "$APP_DB_ADMIN_USER" -d "$APP_DB_NAME" \
  -c "
SELECT
    public_id,
    status,
    attempt,
    failure_code,
    request_id
FROM registrations
ORDER BY submitted_at;
"
              public_id               |  status  | attempt |  failure_code  |              request_id
--------------------------------------+----------+---------+----------------+---------------------------------------
 192d50b8-0fdc-4f49-82b4-461efea4dd55 | VERIFIED |       1 |                | REQ-b5823bafa0c844a6a82a71114a1ab841
 a341e831-c51e-4d57-b435-d1f3cf8074f9 | FAILED   |       3 | VERIFY-TIMEOUT | REQ-2a8478a049be4888abbb879e680f91c6
(2 rows)
```

Gambar 35 Database mencatat transaksi Baseline dan insiden

Perintah pada Gambar 35 menampilkan status transaksi yang tersimpan di app-db. Terlihat transaksi Baseline VERIFIED attempt 1 dan transaksi insiden FAILED attempt 3 dengan failure_code VERIFY-TIMEOUT. Request ID insiden adalah REQ-2a8478a049be4888abbb879e680f91c6.

4) Menganalisis kronologi centralized log tanpa membaca kode aplikasi

```
qasasi@IbramQSS:~/docker-lab/pmb-devsecops-lab$ docker compose exec -T log-db \
  psql -U "$LOG_DB_ADMIN_USER" -d "$LOG_DB_NAME" \
  -c "
WITH events AS (
    SELECT time, (data ->> 'log')::jsonb AS event
    FROM logs.fluentbit
    WHERE tag LIKE 'pmb.%'
      AND left(ltrim(data ->> 'log'), 1) = '{'
)
SELECT
    time,
    event ->> 'service' AS service,
    event ->> 'event' AS event_name,
    event ->> 'severity' AS severity,
    event ->> 'attempt' AS attempt,
    event ->> 'error_code' AS error_code
FROM events
WHERE event ->> 'request_id' = 'REQ-2a8478a049be4888abbb879e680f91c6'
ORDER BY time;
"
        time         |       service       |       event_name       | severity | attempt |   error_code
---------------------+---------------------+------------------------+----------+---------+----------------
 2026-10-28 07:27:33 | registration-api    | registration_submitted | INFO     |         |
 2026-10-28 07:27:33 | registration-api    | http_request           | INFO     |         |
 2026-10-28 07:27:33 | verification-worker | verification_started   | INFO     |         |
 2026-10-28 07:27:33 | verification-worker | verification_retry     | WARN     | 1       |
 2026-10-28 07:27:34 | verification-worker | verification_retry     | WARN     | 2       |
 2026-10-28 07:27:35 | verification-worker | verification_retry     | WARN     | 3       |
 2026-10-28 07:27:36 | verification-worker | dependency_timeout     | ERROR    | 3       | VERIFY-TIMEOUT
(7 rows)
```

Gambar 36 Urutan tujuh event selama insiden

Perintah pada Gambar 36 mengambil event dari log-db menggunakan Request ID transaksi gagal. Terlihat urutan registration_submitted, http_request, verification_started, tiga verification_retry berlevel WARN (attempt 1–3), lalu dependency_timeout berlevel ERROR dengan kode VERIFY-TIMEOUT. Rentang timestamp sekitar 07:27:33–07:27:36 UTC.

5) Menentukan service pertama yang mencatat WARN atau ERROR

```
qasasi@IbramQSS:~/docker-lab/pmb-devsecops-lab$ docker compose exec -T log-db \
  psql -U "$LOG_DB_ADMIN_USER" -d "$LOG_DB_NAME" \
  -c "
WITH events AS (
    SELECT time, (data ->> 'log')::jsonb AS event
    FROM logs.fluentbit
    WHERE tag LIKE 'pmb.%'
      AND left(ltrim(data ->> 'log'), 1) = '{'
)
SELECT
    time,
    event ->> 'service' AS service,
    event ->> 'event' AS event_name,
    event ->> 'severity' AS severity,
    event ->> 'error_code' AS error_code
FROM events
WHERE event ->> 'request_id' = 'REQ-2a8478a049be4888abbb879e680f91c6'
  AND event ->> 'severity' IN ('WARN', 'ERROR')
ORDER BY time;
"
        time         |       service       |     event_name     | severity |   error_code
---------------------+---------------------+--------------------+----------+----------------
 2026-10-28 07:27:33 | verification-worker | verification_retry | WARN     |
 2026-10-28 07:27:34 | verification-worker | verification_retry | WARN     |
 2026-10-28 07:27:35 | verification-worker | verification_retry | WARN     |
 2026-10-28 07:27:36 | verification-worker | dependency_timeout | ERROR    | VERIFY-TIMEOUT
(4 rows)
```

Gambar 37 Identifikasi sumber peringatan dan kesalahan

Perintah pada Gambar 37 menyaring severity WARN dan ERROR untuk transaksi insiden. Event WARN pertama berasal dari verification-worker pada verification_retry; service yang sama kemudian mencatat dependency_timeout berlevel ERROR. Hasil ini memperlihatkan bahwa gangguan terdeteksi di tahap verifikasi, bukan ketika registration-api menerima pendaftaran.

F. Tahap C — Recovery

1) Mengembalikan mode insiden menjadi none

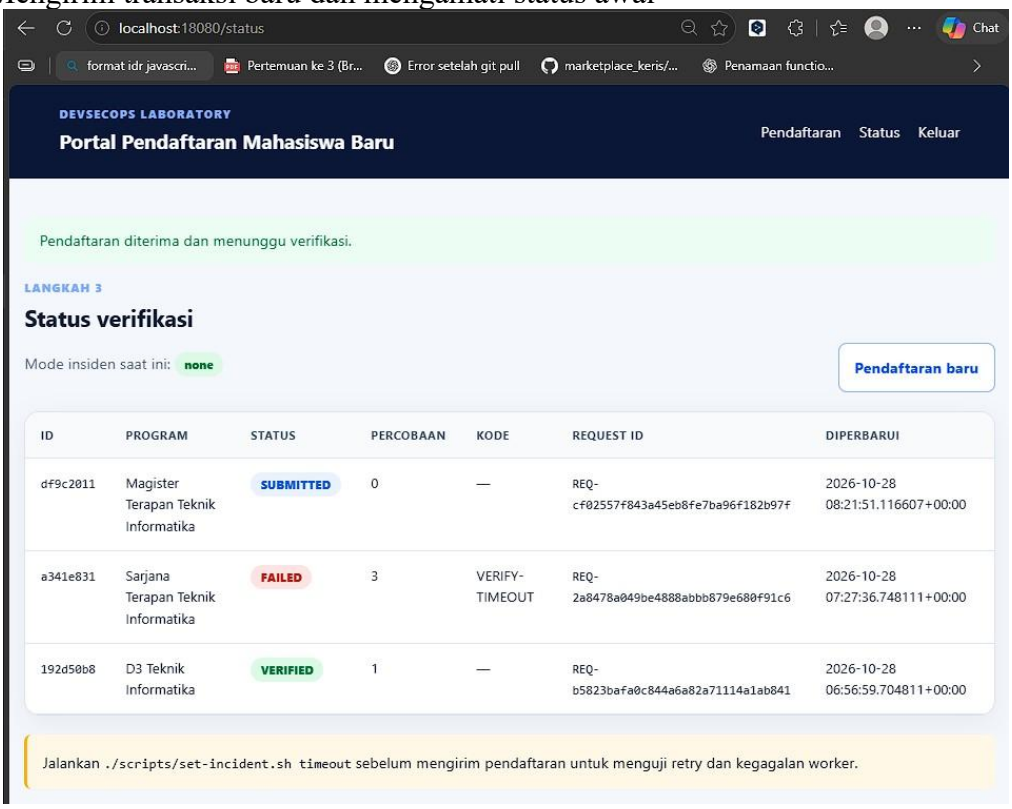
```
qasasi@IbramQSS:~/docker-lab/pmb-devsecops-lab$ ./scripts/set-incident.sh none
UPDATE 1
      key       | value |          updated_at
----------------+-------+-------------------------------
 incident_mode | none  | 2026-10-28 08:15:47.664045+00
(1 row)

[PASS] Mode insiden: none
```

Gambar 38 Menonaktifkan kembali simulasi timeout

Perintah pada Gambar 38 mengembalikan incident_mode ke none melalui set-incident.sh. Output [PASS] Mode insiden: none menunjukkan bahwa pengujian selanjutnya menggunakan kondisi normal.

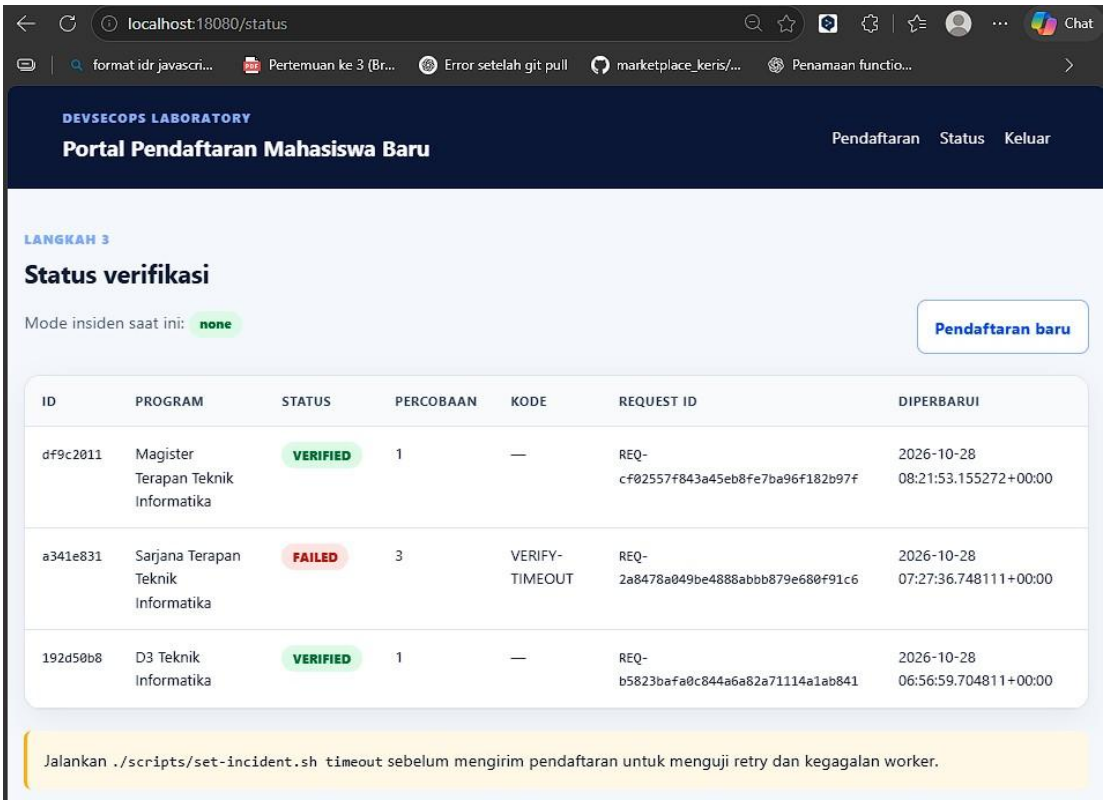
2) Mengirim transaksi baru dan mengamati status awal



Gambar 39 Transaksi baru masih berstatus SUBMITTED

Gambar 39 menunjukkan transaksi pemulihan telah diterima sebagai SUBMITTED dengan percobaan 0. Transaksi baru ini berbeda dari transaksi pada Tahap B yang sudah berstatus FAILED.

3) Memastikan verifikasi baru berhasil



Gambar 40 Transaksi baru VERIFIED tanpa mengubah status lama

Gambar 40 menunjukkan transaksi Recovery menjadi VERIFIED pada percobaan 1, sedangkan transaksi timeout sebelumnya tetap FAILED. Penonaktifan mode insiden memulihkan pemrosesan berikutnya, tetapi tidak otomatis menjadwalkan ulang transaksi yang telah mencapai status gagal.

4) Membandingkan hasil ketiga transaksi

```
qasasi@IbramQSS:~/docker-lab/pmb-devsecops-lab$ docker compose exec -T app-db \
  psql -U "$APP_DB_ADMIN_USER" -d "$APP_DB_NAME" \
  -c "
SELECT
    public_id,
    status,
    attempt,
    failure_code,
    request_id
FROM registrations
ORDER BY submitted_at;
"
              public_id               |  status  | attempt |  failure_code  |              request_id
--------------------------------------+----------+---------+----------------+---------------------------------------
 192d50b8-0fdc-4f49-82b4-461efea4dd55 | VERIFIED |       1 |                | REQ-b5823bafa0c844a6a82a71114a1ab841
 a341e831-c51e-4d57-b435-d1f3cf8074f9 | FAILED   |       3 | VERIFY-TIMEOUT | REQ-2a8478a049be4888abbb879e680f91c6
 df9c2011-cad1-417b-96e8-ea54015daa38 | VERIFIED |       1 |                | REQ-cf02557f843a45eb8fe7ba96f182b97f
(3 rows)
```

Gambar 41 Perbandingan database Tahap A, B, dan C

Perintah pada Gambar 41 menampilkan tiga transaksi: Baseline VERIFIED (1 percobaan), Failure Injection FAILED (3 percobaan, VERIFY-TIMEOUT), dan Recovery VERIFIED (1 percobaan). Hasil ini membuktikan keberhasilan pemulihan untuk transaksi baru tanpa mengubah histori transaksi lama.

5) Memeriksa kronologi event transaksi Recovery

```
qasasi@IbramQSS:~/docker-lab/pmb-devsecops-lab$ docker compose exec -T log-db   psql -U "$LOG_DB_ADMIN_USER" -d "$LOG_DB_NAME"   -c "
WITH events AS (
    SELECT time, (data ->> 'log')::jsonb AS event
    FROM logs.fluentbit
    WHERE tag LIKE 'pmb.%'
      AND left(ltrim(data ->> 'log'), 1) = '{'
)
SELECT
    time,
    event ->> 'service' AS service,
    event ->> 'event' AS event_name,
    event ->> 'severity' AS severity,
    event ->> 'attempt' AS attempt
FROM events
WHERE event ->> 'request_id' = 'REQ-cf02557f843a45eb8fe7ba96f182b97f'
ORDER BY time;
"
        time         |       service       |       event_name       | severity | attempt
---------------------+---------------------+------------------------+----------+---------
 2026-10-28 08:21:51 | registration-api    | registration_submitted | INFO     |
 2026-10-28 08:21:51 | registration-api    | http_request           | INFO     |
 2026-10-28 08:21:52 | verification-worker | verification_started   | INFO     |
 2026-10-28 08:21:53 | verification-worker | verification_completed | INFO     | 1
(4 rows)
```

Gambar 42 Empat event INFO setelah pemulihan

Perintah pada Gambar 42 menyaring log berdasarkan Request ID Recovery REQ-cf02557f843a45eb8fe7ba96f182b97f. Tercatat registration_submitted, http_request, verification_started, dan verification_completed, seluruhnya INFO, tanpa verification_retry maupun dependency_timeout.

Transaksi lama tidak otomatis berubah dari FAILED karena perubahan incident_mode hanya memengaruhi kondisi pemrosesan berikutnya; sistem tidak menjalankan ulang pekerjaan yang telah mencapai status gagal. Untuk produksi, transaksi gagal sementara dapat diproses ulang secara terkontrol dengan batas percobaan, exponential backoff, dan idempotensi. Transaksi yang tetap gagal setelah batas retry dapat dipindahkan ke dead-letter queue untuk ditinjau serta diproses ulang oleh operator. Strategi tersebut merupakan usulan, bukan fitur yang telah diterapkan dalam praktikum.

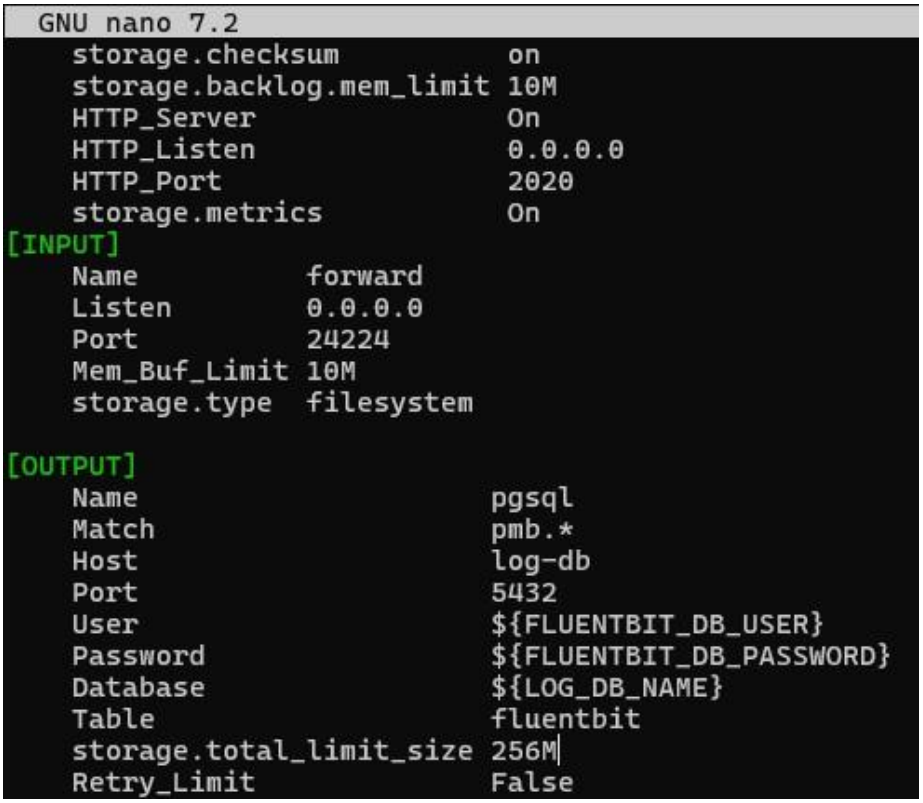
G. Tahap D — Resilience Logging

1) Mencadangkan dan menyesuaikan konfigurasi Fluent Bit

```
qasasi@IbramQSS:~/docker-lab/pmb-devsecops-lab$ cp fluent-bit/fluent-bit.conf fluent-bit/fluent-bit.conf.backup
qasasi@IbramQSS:~/docker-lab/pmb-devsecops-lab$
```

Gambar 43 Membuat cadangan fluent-bit.conf

Perintah pada Gambar 43 menyalin file fluent-bit.conf menjadi fluent-bit.conf.backup sehingga konfigurasi awal dapat dipulihkan apabila perubahan retry dan buffering menimbulkan masalah.



Gambar 44 Menambahkan storage metrics dan kebijakan retry

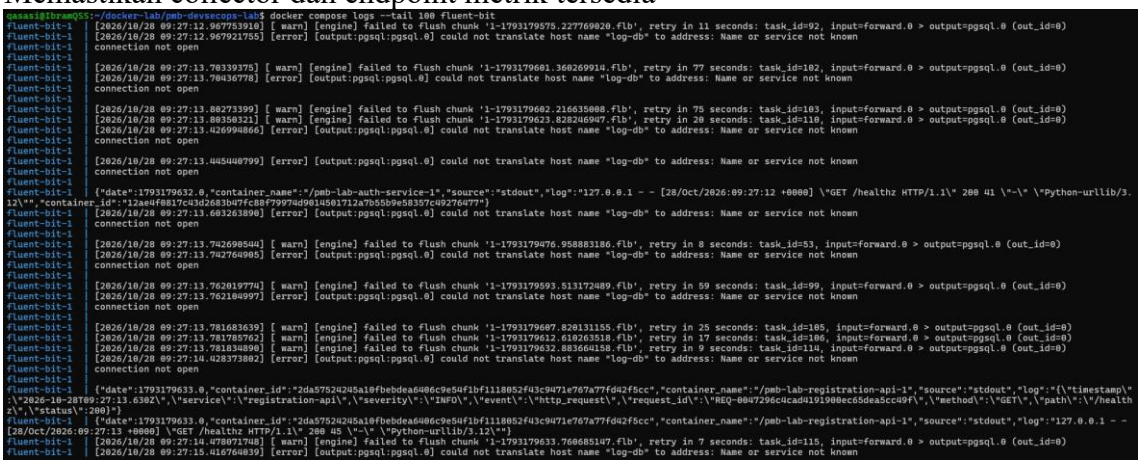
Gambar 44 memperlihatkan konfigurasi Fluent Bit yang memakai filesystem storage dan batas antrean output 256M. Pengaturan storage.metrics On digunakan untuk memantau buffer, sedangkan Retry_Limit False mempertahankan upaya pengiriman ulang selama chunk masih tersedia dan kapasitas penyimpanan mencukupi.



Gambar 45 Menjalankan ulang container Fluent Bit

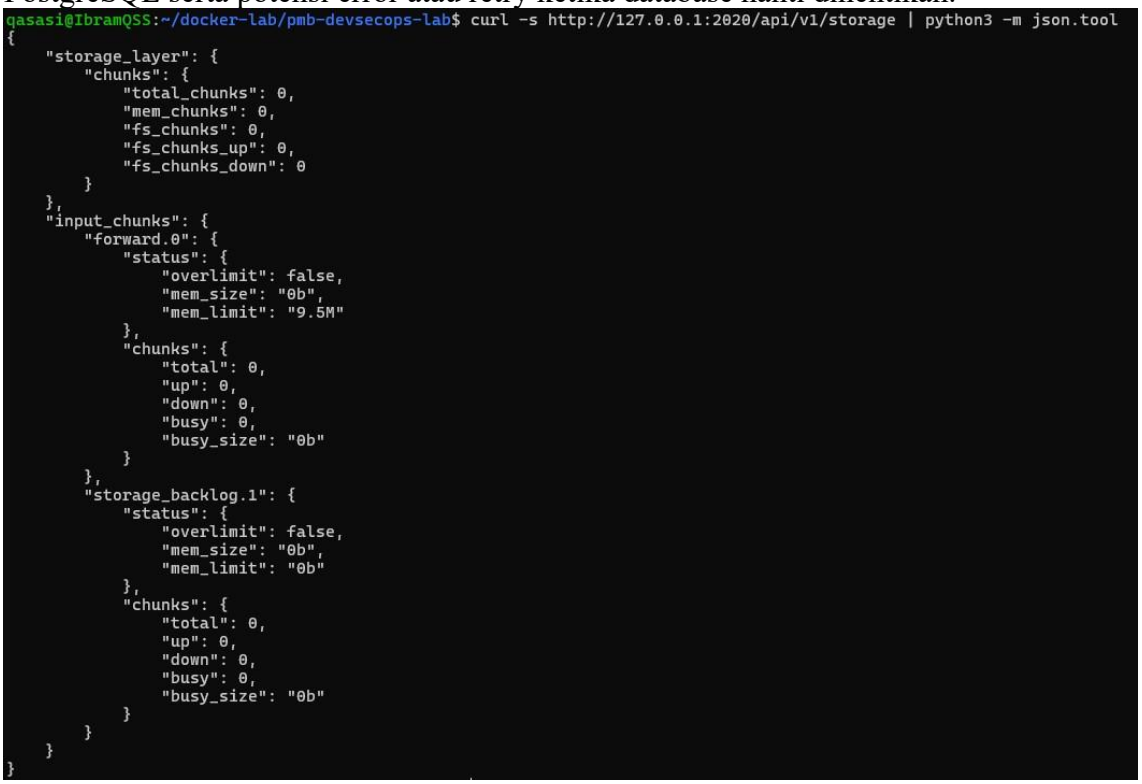
Perintah pada Gambar 45 menerapkan konfigurasi baru melalui docker compose up -d --no-deps --force-recreate fluent-bit. Hasilnya container Fluent Bit berhasil di-recreate dan dijalankan tanpa mengulang seluruh service lainnya.

2) Memastikan collector dan endpoint metrik tersedia



Gambar 46 Memeriksa aktivitas container Fluent Bit

Perintah pada Gambar 46 digunakan untuk memeriksa log container Fluent Bit yang sedang berjalan. Keluaran menjadi dasar pemeriksaan aktivitas output PostgreSQL serta potensi error atau retry ketika database nanti dihentikan.



Gambar 47 Memeriksa endpoint storage sebelum gangguan

Perintah pada Gambar 47 mengakses endpoint HTTP /api/v1/storage melalui port 2020. Nilai total_chunks 0 menunjukkan tidak terdapat antrean aktif pada saat pemeriksaan awal, sekaligus membuktikan endpoint metrik dapat diakses.

3) Mencatat jumlah log sebelum database dihentikan

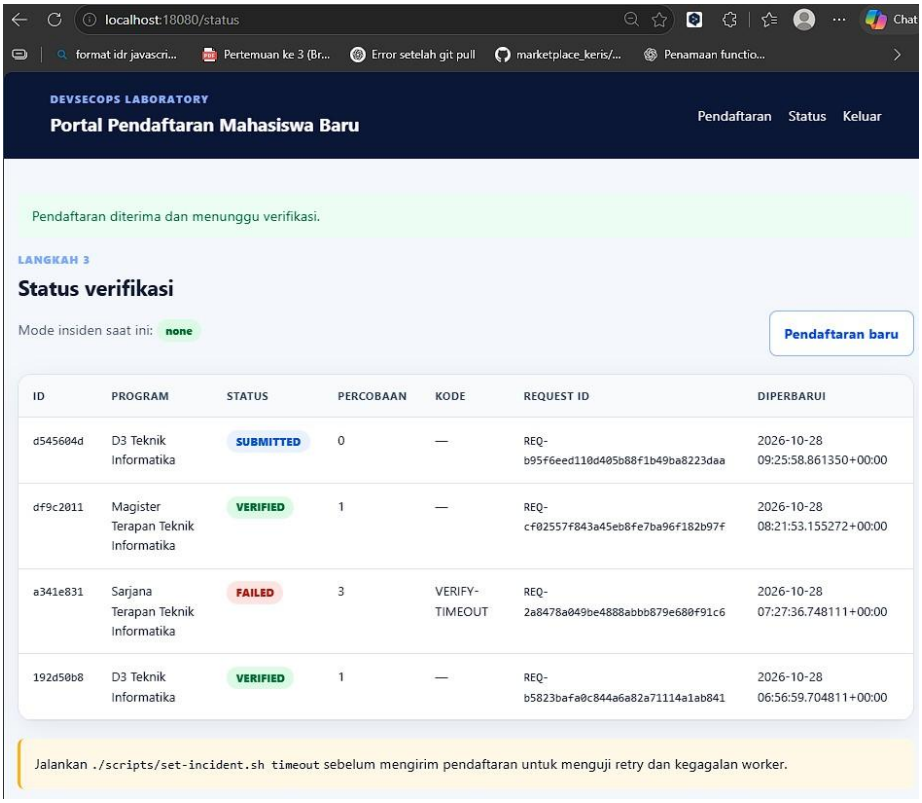
```
qasasi@IbramQSS:~/docker-lab/pmb-devsecops-lab$ docker compose exec -T log-db \
  psql -U "$LOG_DB_ADMIN_USER" -d "$LOG_DB_NAME" \
  -c "
SELECT COUNT(*) AS total_log_sebelum
FROM logs.fluentbit;
"
 total_log_sebelum
-------------------
              4220
(1 row)
```

Gambar 48 Jumlah log awal pada database log

Perintah pada Gambar 48 menghitung seluruh record pada tabel logs.fluentbit sebelum simulasi gangguan. Nilai awal yang diperoleh adalah 4.220 record, untuk dibandingkan dengan kondisi setelah pemulihan.

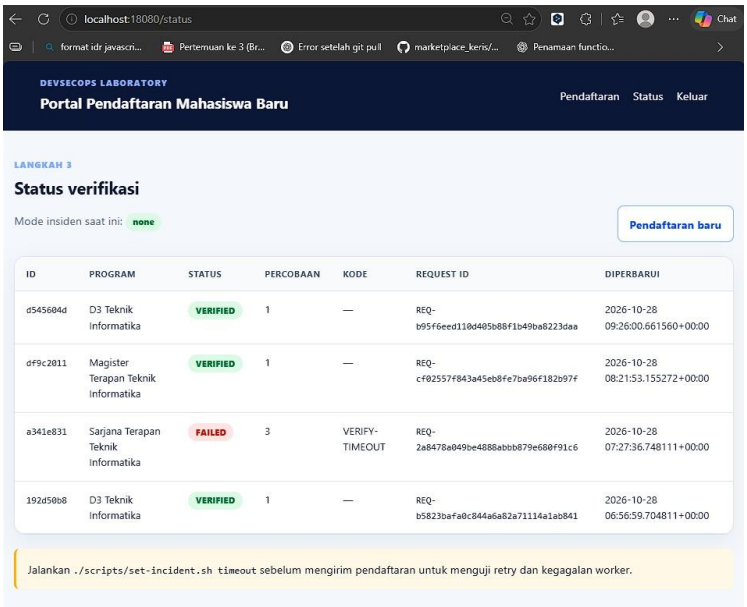
4) Menghentikan log-db sementara

```
qasasi@IbramQSS:~/docker-lab/pmb-devsecops-lab$ docker compose stop log-db
[+] stop 1/1
 ✔Container pmb-lab-log-db-1 Stopped                                                
```



Gambar 51 Status awal SUBMITTED ketika log-db berhenti

Gambar 51 memperlihatkan transaksi baru masih SUBMITTED. Walaupun database log dimatikan, registration-api tetap dapat menerima transaksi pada app-db.



Gambar 52 Status VERIFIED meskipun log-db tidak tersedia

Gambar 52 menunjukkan transaksi baru menjadi VERIFIED pada percobaan 1, sementara mode insiden tetap none. Ini membuktikan bahwa kegagalan backend logging tidak langsung menghentikan proses bisnis pendaftaran.

6) Mengamati retry dan penggunaan buffer saat gangguan

```
qasasi@IbramQSS:~/docker-lab/pmb-devsecops-lab$ curl -s http://127.0.0.1:2020/api/v1/metrics | python3 -m json.tool
{
    "input": {
        "forward.0": {
            "records": 348,
            "bytes": 106815
        },
        "storage_backlog.1": {
            "records": 0,
            "bytes": 0
        }
    },
    "filter": {},
    "output": {
        "pgsql.0": {
            "proc_records": 118,
            "proc_bytes": 35522,
            "errors": 0,
            "retries": 724,
            "retries_failed": 0,
            "dropped_records": 0,
            "retried_records": 1160
        },
        "stdout.1": {
            "proc_records": 324,
            "proc_bytes": 99583,
            "errors": 0,
            "retries": 0,
            "retries_failed": 0,
            "dropped_records": 0,
            "retried_records": 0
        }
    }
}
```

Gambar 53 Metrik retry Fluent Bit selama gangguan

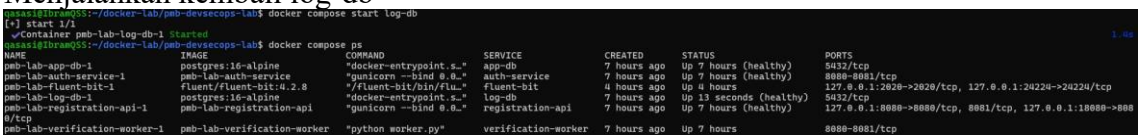
Perintah pada Gambar 53 membaca /api/v1/metrics. Output pgsql mencatat 724 retries, 0 retries_failed, dan 0 dropped_records pada pengamatan tersebut. Metrik ini bersifat kumulatif selama proses Fluent Bit aktif.

```
qasasi@IbramQSS:~/docker-lab/pmb-devsecops-lab$ curl -s http://127.0.0.1:2020/api/v1/storage | python3 -m json.tool
{
    "storage_layer": {
        "chunks": {
            "total_chunks": 130,
            "mem_chunks": 0,
            "fs_chunks": 130,
            "fs_chunks_up": 128,
            "fs_chunks_down": 2
        }
    },
    "input_chunks": {
        "forward.0": {
            "status": {
                "overlimit": false,
                "mem_size": "62.6K",
                "mem_limit": "9.5M"
            },
            "chunks": {
                "total": 130,
                "up": 128,
                "down": 2,
                "busy": 128,
                "busy_size": "62.6K"
            }
        },
        "storage_backlog.1": {
            "status": {
                "overlimit": false,
                "mem_size": "0b",
                "mem_limit": "0b"
            },
            "chunks": {
                "total": 0,
                "up": 0,
                "down": 0,
                "busy": 0,
                "busy_size": "0b"
            }
        }
    }
}
```

Gambar 54 Buffer filesystem menampung 130 chunk

Perintah pada Gambar 54 membaca /api/v1/storage ketika backend logging berhenti. Tercatat 130 filesystem chunk, 128 chunk up, dan 2 chunk down; data belum dapat seluruhnya dikirim ke PostgreSQL sehingga ditampung sementara.

7) Menjalankan kembali log-db



Gambar 55 Memulihkan backend log-db

Perintah pada Gambar 55 menjalankan kembali log-db kemudian memeriksa status container. Hasilnya log-db kembali Up dan healthy, sehingga Fluent Bit dapat mencoba mengirim chunk yang tertunda.

8) Memeriksa pertambahan record dan kondisi awal setelah pemulihan

```
qasasi@IbramQSS:~/docker-lab/pmb-devsecops-lab$ docker compose exec -T log-db \
  psql -U "$LOG_DB_ADMIN_USER" -d "$LOG_DB_NAME" \
  -c "
SELECT COUNT(*) AS total_log_setelah
FROM logs.fluentbit;
"
 total_log_setelah
-------------------
              5302
(1 row)
```

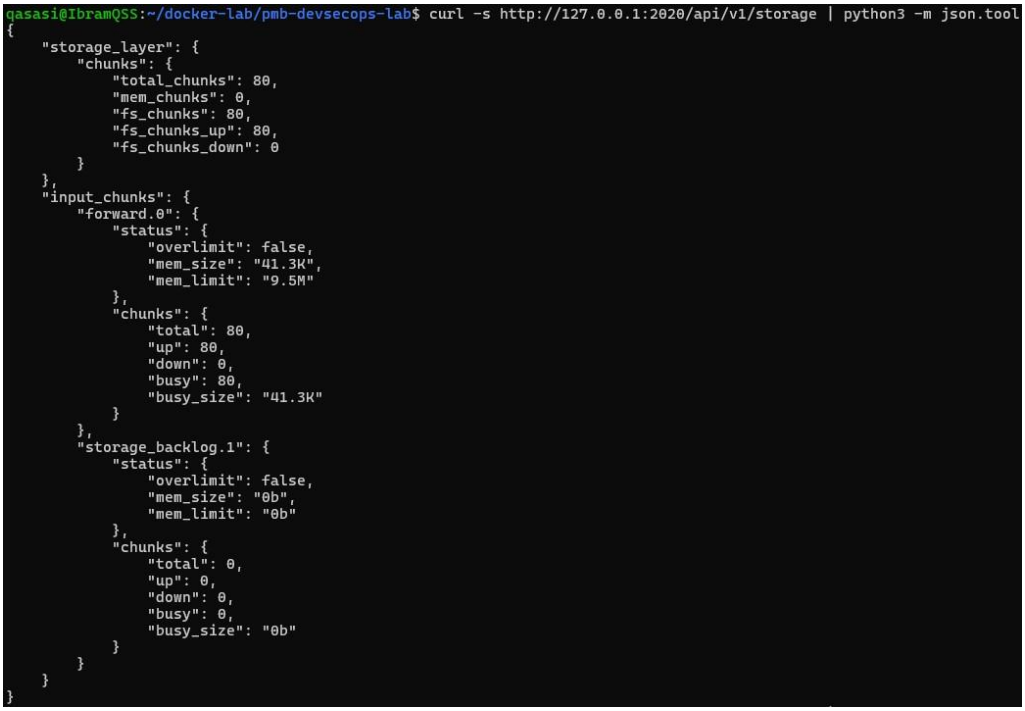
Gambar 56 Jumlah log setelah database kembali aktif

Perintah pada Gambar 56 mencatat 5.302 record setelah pemulihan, bertambah 1.082 dari sebelum gangguan. Pertambahan ini sendiri belum membuktikan seluruh backlog terkirim, karena log healthcheck dan aktivitas lain juga ikut bertambah.

```
qasasi@IbramQSS:~/docker-lab/pmb-devsecops-lab$ docker compose exec -T log-db \
  psql -U "$LOG_DB_ADMIN_USER" -d "$LOG_DB_NAME" \
  -c "
WITH events AS (
    SELECT
        time,
        (data ->> 'log')::jsonb AS event
    FROM logs.fluentbit
    WHERE tag LIKE 'pmb.%'
      AND left(ltrim(data ->> 'log'), 1) = '{'
)
SELECT
    time,
    event ->> 'service' AS service,
    event ->> 'event' AS event_name,
    event ->> 'severity' AS severity,
    event ->> 'attempt' AS attempt
FROM events
WHERE event ->> 'request_id' =
'REQ-b95f6eed110d405b88f1b49ba8223daa'
ORDER BY time;
"
 time | service | event_name | severity | attempt
------+---------+------------+----------+---------
(0 rows)
```

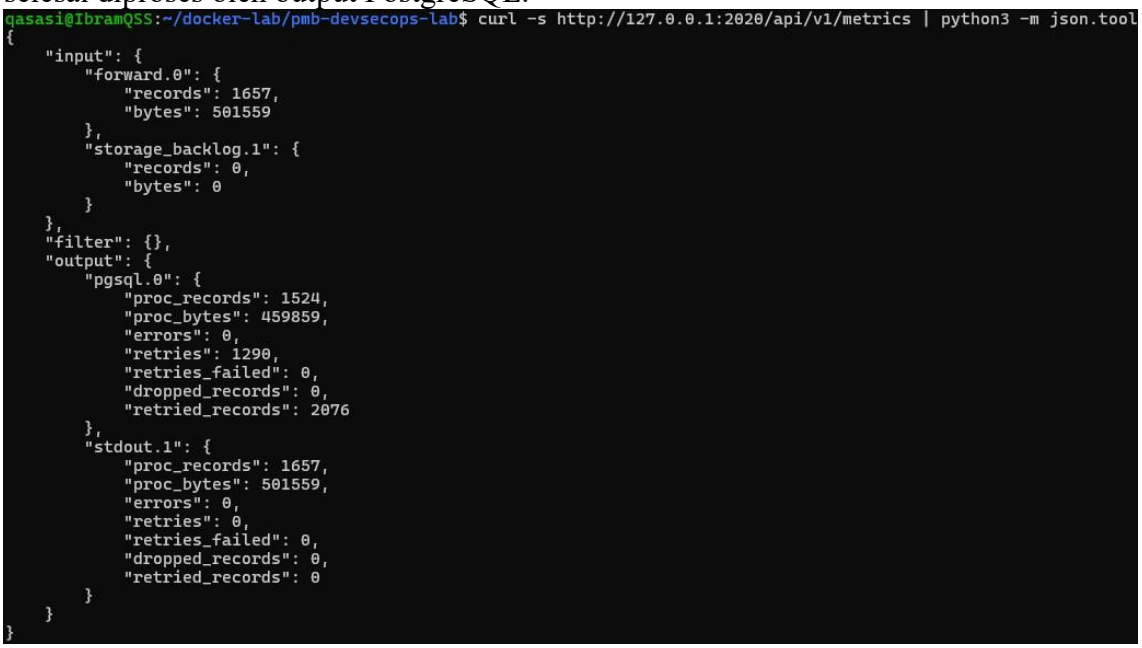
Gambar 57 Pemeriksaan pertama log transaksi Tahap D

Perintah pada Gambar 57 menelusuri Request ID REQ-b95f6eed110d405b88f1b49ba8223daa ketika database baru dipulihkan. Hasil awal 0 rows menunjukkan keempat event belum langsung ditemukan, sehingga pengiriman backlog perlu diperiksa lebih lanjut.



Gambar 58 Buffer masih berisi 80 filesystem chunk

Gambar 58 memperlihatkan buffer belum kosong setelah log-db aktif kembali: tercatat 80 filesystem chunk. Ini menunjukkan masih ada antrean yang belum selesai diproses oleh output PostgreSQL.



Gambar 59 Metrik pengiriman setelah log-db aktif

Perintah pada Gambar 59 menunjukkan proc_records output pgsql telah meningkat menjadi 1.524, retries 1.290, retries_failed 0 dan dropped_records 0. Hasil ini menunjukkan pemrosesan log berlanjut tanpa pembuangan record yang dilaporkan dalam snapshot tersebut.

9) Menelusuri event yang masuk bertahap

```
qasasi@IbramQSS:~/docker-lab/pmb-devsecops-lab$ docker compose exec -T log-db \
  psql -U "$LOG_DB_ADMIN_USER" -d "$LOG_DB_NAME" \
  -c "
SELECT
    time,
    (data ->> 'log')::jsonb ->> 'service' AS service,
    (data ->> 'log')::jsonb ->> 'event' AS event_name,
    (data ->> 'log')::jsonb ->> 'severity' AS severity
FROM logs.fluentbit
WHERE data ->> 'log' LIKE
'%REQ-b95f6eed110d405b88f1b49ba8223daa%'
ORDER BY time;
"
        time         |       service       |       event_name       | severity
---------------------+---------------------+------------------------+----------
 2026-10-28 09:26:00 | verification-worker | verification_completed | INFO
(1 row)
```

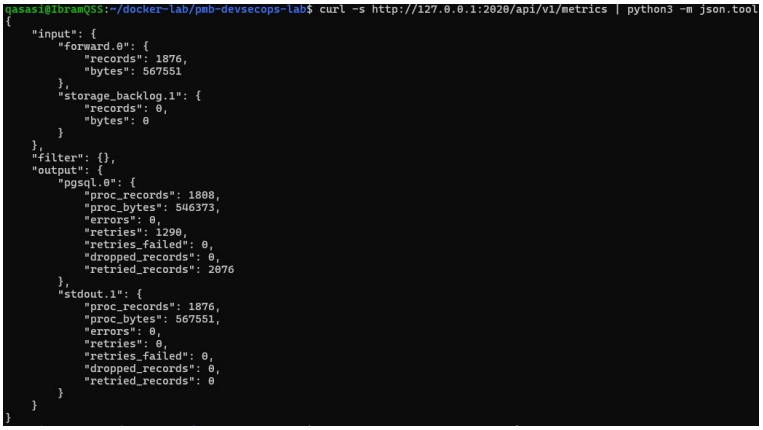
Gambar 60 Event verification_completed ditemukan pertama kali

Perintah pada Gambar 60 mencari Request ID secara langsung melalui LIKE di kolom pesan log. Hasil query menemukan satu event verification_completed dari verification-worker, sedangkan event lain belum ditemukan saat pemeriksaan tersebut.

```
qasasi@IbramQSS:~/docker-lab/pmb-devsecops-lab$ curl -s http://127.0.0.1:2020/api/v1/storage | python3 -m json.tool
{
    "storage_layer": {
        "chunks": {
            "total_chunks": 44,
            "mem_chunks": 0,
            "fs_chunks": 44,
            "fs_chunks_up": 44,
            "fs_chunks_down": 0
        }
    },
    "input_chunks": {
        "forward.0": {
            "status": {
                "overlimit": false,
                "mem_size": "22.2K",
                "mem_limit": "9.5M"
            },
            "chunks": {
                "total": 44,
                "up": 44,
                "down": 0,
                "busy": 44,
                "busy_size": "22.2K"
            }
        },
        "storage_backlog.1": {
            "status": {
                "overlimit": false,
                "mem_size": "0b",
                "mem_limit": "0b"
            },
            "chunks": {
                "total": 0,
                "up": 0,
                "down": 0,
                "busy": 0,
                "busy_size": "0b"
            }
        }
    }
}
```

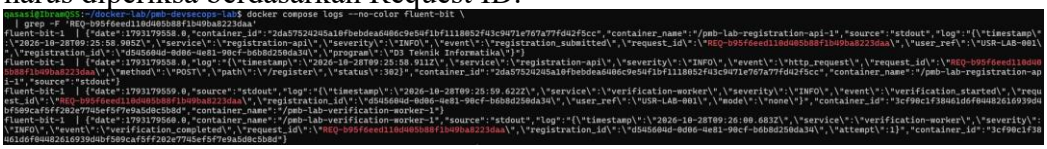
Gambar 61 Buffer berkurang menjadi 80 chunk aktif

Gambar 61 menunjukkan pemeriksaan filesystem buffer pada tahap pengiriman ulang. Nilai total_chunks 80 menandakan backlog belum habis walaupun database sudah pulih.



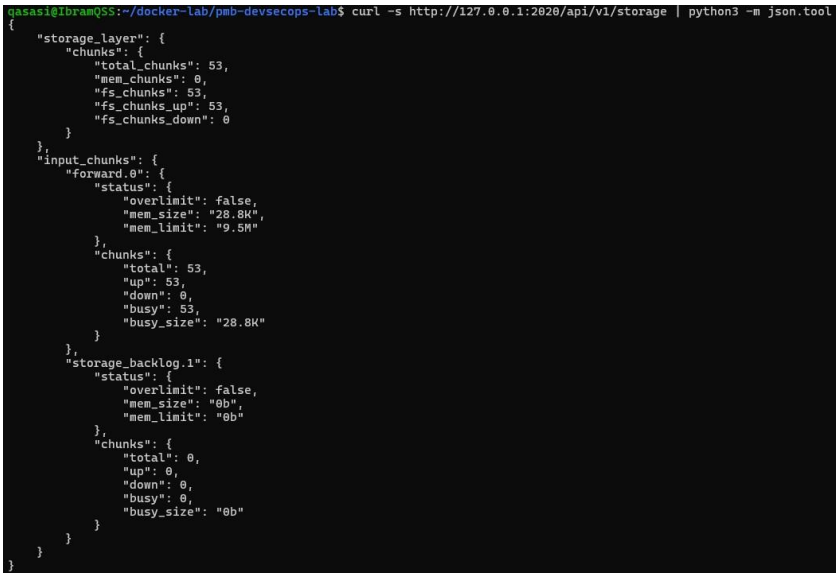
Gambar 62 Jumlah record diproses output meningkat

Gambar 62 memperlihatkan metrik output PostgreSQL saat pemulihan berlangsung. Jumlah proc_records meningkat dan dropped_records tetap nol sehingga pengiriman sedang berproses, meskipun kelengkapan transaksi tetap harus diperiksa berdasarkan Request ID.



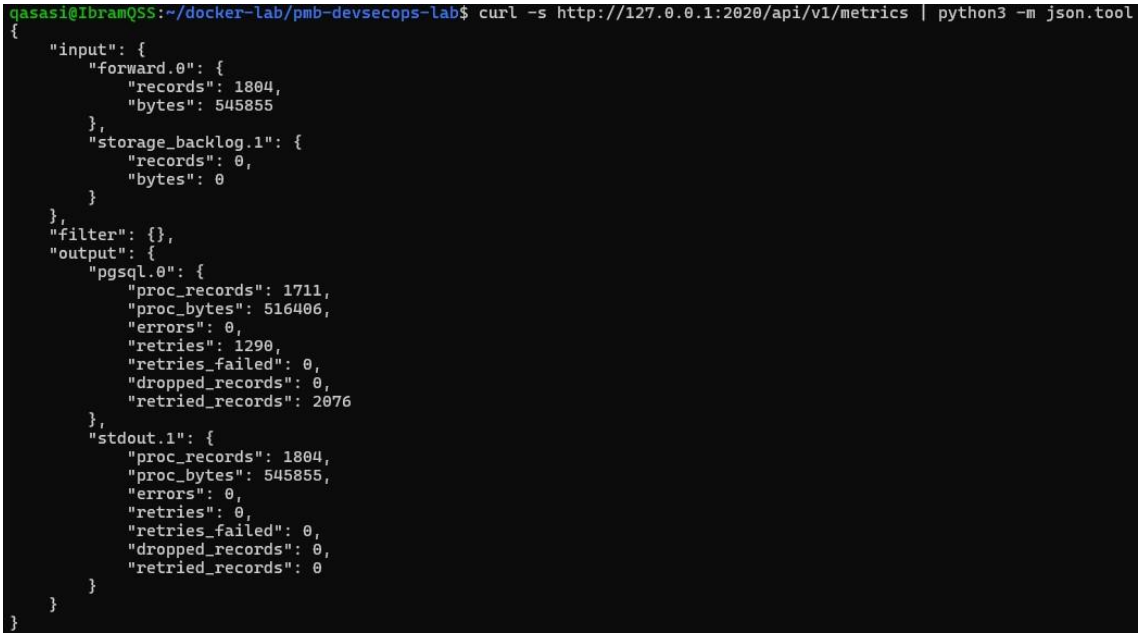
Gambar 63 Bukti collector menerima empat event transaksi

Perintah pada Gambar 63 mencari Request ID pada stdout Fluent Bit. Keempat event registration_submitted, http_request, verification_started, dan verification_completed berhasil ditemukan di sisi collector. Dengan demikian setiap event telah mencapai Fluent Bit sebelum diperiksa kelengkapannya di database.



Gambar 64 Jumlah filesystem chunk turun menjadi 53

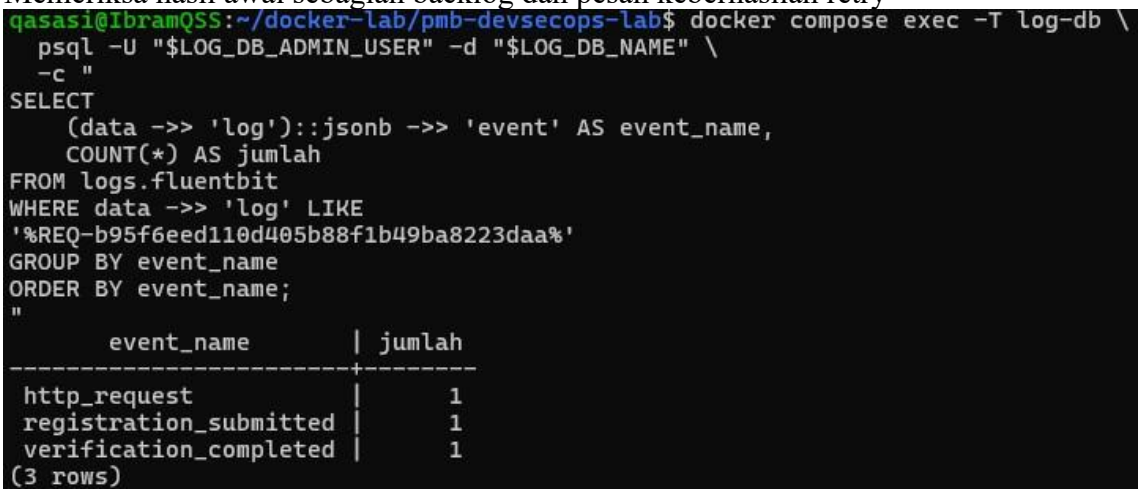
Gambar 64 menunjukkan buffer filesystem kembali berkurang hingga 53 chunk. Penurunan ini konsisten dengan proses pengosongan antrean sesudah pemulihan database.



Gambar 65 Record berhasil diproses bertambah

Gambar 65 menunjukkan proc_records output PostgreSQL meningkat menjadi 1.711, sedangkan retry tetap 1.290 dan dropped_records 0. Snapshot ini memperkuat bukti bahwa proses pengiriman kembali berjalan.

10) Memeriksa hasil awal sebagian backlog dan pesan keberhasilan retry



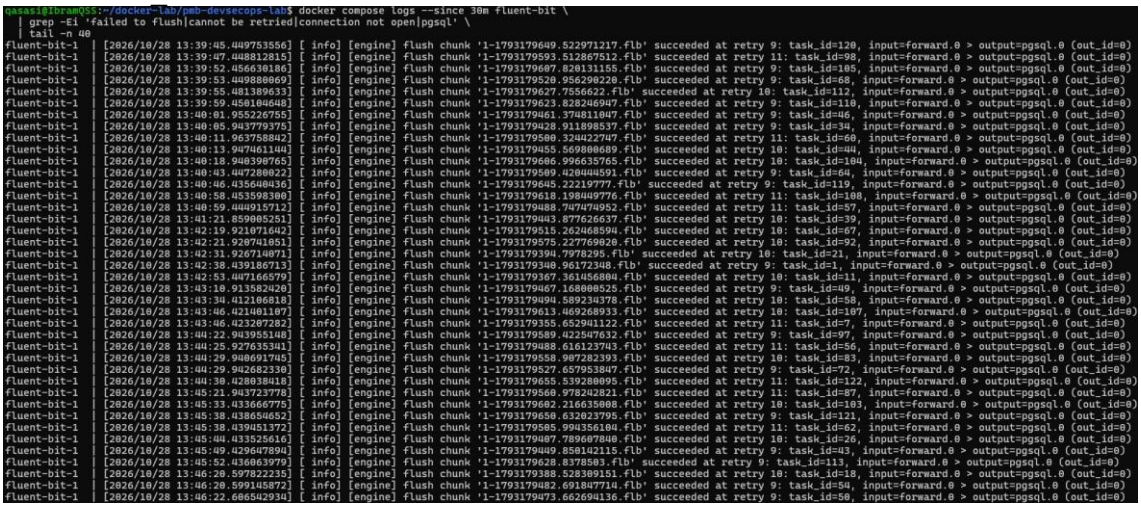
Gambar 66 Tiga event transaksi terdeteksi di log-db

Perintah pada Gambar 66 mengelompokkan event berdasarkan jenisnya. Tiga event ditemukan masing-masing satu kali; verification_started belum ada pada pemeriksaan tersebut, sehingga pemulihan seluruh log transaksi belum bisa dinyatakan lengkap.

```
qasasi@IbramQSS:~/docker-lab/pmb-devsecops-lab$ docker compose exec -T log-db \
  psql -U "$LOG_DB_ADMIN_USER" -d "$LOG_DB_NAME" \
  -c "
SELECT
    time,
    (data ->> 'log')::jsonb ->> 'service' AS service,
    (data ->> 'log')::jsonb ->> 'event' AS event_name,
    (data ->> 'log')::jsonb ->> 'severity' AS severity
FROM logs.fluentbit
WHERE data ->> 'log' LIKE
'%REQ-b95f6eed110d405b88f1b49ba8223daa%'
ORDER BY time;
"
        time         |       service       |        event_name         | severity
---------------------+---------------------+---------------------------+----------
 2026-10-28 09:25:58 | registration-api    | registration_submitted    | INFO
 2026-10-28 09:25:58 | registration-api    | http_request              | INFO
 2026-10-28 09:26:00 | verification-worker | verification_completed    | INFO
(3 rows)
```

Gambar 67 Pemeriksaan kronologi saat baru tiga event tersimpan

Gambar 67 menampilkan kronologi dari log-db berdasarkan Request ID. Tiga event yang sudah tersimpan adalah registration_submitted, http_request, dan verification_completed; urutan verifikasi belum lengkap karena verification_started belum muncul.



Gambar 68 Bukti chunk berhasil di-flush setelah retry

Perintah pada Gambar 68 memeriksa log Fluent Bit dan menampilkan pesan flush chunk succeeded at retry 9 hingga retry 11. Pesan ini merupakan bukti langsung bahwa sejumlah chunk yang sebelumnya gagal dikirim akhirnya berhasil diproses setelah database aktif kembali.

11) Melanjutkan pemantauan hingga empat event berhasil tersimpan

```
qasasi@IbramQSS:~/docker-lab/pmb-devsecops-lab$ curl -s http://127.0.0.1:2020/api/v1/storage | python3 -m json.tool
{
    "storage_layer": {
        "chunks": {
            "total_chunks": 29,
            "mem_chunks": 0,
            "fs_chunks": 29,
            "fs_chunks_up": 29,
            "fs_chunks_down": 0
        }
    },
    "input_chunks": {
        "forward.0": {
            "status": {
                "overlimit": false,
                "mem_size": "14.3K",
                "mem_limit": "9.5M"
            },
            "chunks": {
                "total": 29,
                "up": 29,
                "down": 0,
                "busy": 29,
                "busy_size": "14.3K"
            }
        },
        "storage_backlog.1": {
            "status": {
                "overlimit": false,
                "mem_size": "0b",
                "mem_limit": "0b"
            },
            "chunks": {
                "total": 0,
                "up": 0,
                "down": 0,
                "busy": 0,
                "busy_size": "0b"
            }
        }
    }
}
```

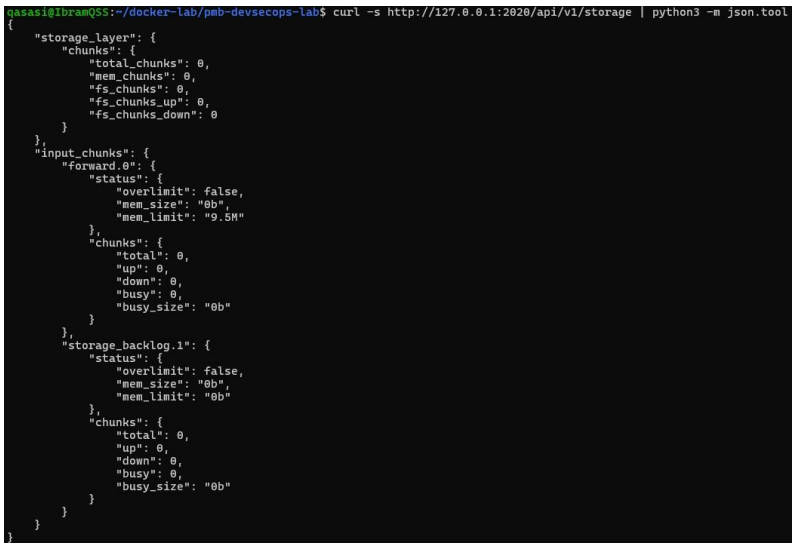
Gambar 69 Pemeriksaan ketika 29 chunk masih tersisa

Perintah pada Gambar 69 menunjukkan filesystem buffer masih memiliki 29 chunk dan nilai busy juga 29. Kondisi tersebut berarti antrean belum selesai sepenuhnya walaupun event transaksi terus diperiksa.

```
qasasi@IbramQSS:~/docker-lab/pmb-devsecops-lab$ docker compose exec -T log-db \
  psql -U "$LOG_DB_ADMIN_USER" -d "$LOG_DB_NAME" \
  -c "
WITH expected(event_name) AS (
    VALUES
      ('registration_submitted'),
      ('http_request'),
      ('verification_started'),
      ('verification_completed')
),
actual AS (
    SELECT data ->> 'log' AS message
    FROM logs.fluentbit
    WHERE data ->> 'log' LIKE
      '%REQ-b95f6eed110d405b88f1b49ba8223daa%'
)
SELECT
    e.event_name,
    COUNT(a.message) AS jumlah
FROM expected e
LEFT JOIN actual a
    ON a.message LIKE
       '%\"event\":\"' || e.event_name || '\"%'
GROUP BY e.event_name
ORDER BY e.event_name;
"
       event_name       | jumlah
------------------------+--------
 http_request           |      1
 registration_submitted |      1
 verification_completed |      1
 verification_started   |      1
(4 rows)
```

Gambar 70 Keempat event transaksi akhirnya tersimpan

Perintah pada Gambar 70 menyatukan daftar event yang diharapkan dengan log aktual dan menghitung masing-masing record. Hasilnya keempat event registration_submitted, http_request, verification_started, dan verification_completed masing-masing berjumlah 1. Bukti ini menunjukkan pemulihan backlog berhasil untuk seluruh event transaksi uji tersebut.



Gambar 71 Buffer collector telah kosong

Perintah pada Gambar 71 memeriksa kembali endpoint storage setelah proses retry. Screenshot menunjukkan total_chunks 0 dan fs_chunks 0, yang berarti tidak ada chunk aktif di buffer pada saat pemeriksaan terakhir.

Hasil Tahap D membuktikan bahwa Fluent Bit dapat menampung event pada filesystem buffer ketika PostgreSQL berhenti dan mengirimkan ulang backlog setelah database pulih. Pada saat gangguan buffer mencapai 130 chunk dan metrik mencatat retry, sedangkan setelah pemulihan empat dari empat event transaksi uji ditemukan satu kali dan buffer pada pemeriksaan akhir kosong. Hasil tersebut tidak membuktikan exactly-once delivery secara umum: jika database sudah menyimpan batch tetapi konfirmasi ke Fluent Bit terputus, retry berpotensi menyimpan duplikat; saat storage.total_limit_size 256M terlampaui, chunk tertua pun dapat dibuang. Perlu event_id unik, deduplikasi, dan pemantauan kehilangan atau duplikasi untuk jaminan lebih kuat.

H. Pertanyaan Analitis

1) Mengapa status container Up tidak membuktikan fungsi bisnis sehat?

Status Up hanya menunjukkan proses utama container masih berjalan, bukan bahwa alur bisnis bekerja benar. Pada Tahap B, container aplikasi tetap aktif tetapi transaksi berakhir FAILED akibat VERIFY-TIMEOUT. Kesehatan bisnis perlu diuji melalui endpoint, log error, status pada database transaksi, dan hasil verifikasi end-to-end.

2) Apa perbedaan event time, waktu penerimaan collector, dan waktu penyimpanan database?

Event time adalah waktu kejadian di aplikasi yang tercatat pada field timestamp dalam JSON. Waktu penerimaan collector adalah saat Fluent Bit mendapatkan record melalui input forward; output stdout memuat field date. Waktu penyimpanan database adalah saat record benar-benar disimpan PostgreSQL. Perbedaannya dapat besar ketika terjadi buffering dan retry; tabel lab tidak mempunyai kolom khusus waktu INSERT sehingga waktu penyimpanan tepatnya tidak dapat dipastikan dari field time saja.

3) Mengapa nama dan email tidak dimasukkan ke structured log?

Nama dan email adalah data pribadi yang tidak diperlukan untuk penelusuran teknis setiap transaksi. Pencatatan keduanya memperbesar risiko kebocoran melalui sistem log, akses operator, dan arsip. Korelasi cukup memakai request_id, user_ref, service, event, timestamp, status, serta error_code; data identitas sebaiknya hanya tersimpan pada database transaksi dengan akses terbatas.

4) Apa risiko penggunaan request_id dari client tanpa validasi?

Client dapat mengirim ID palsu, memakai ID transaksi lain, atau memasukkan karakter kontrol dan string terlalu panjang sehingga memicu pencemaran log, salah korelasi, dan kesulitan investigasi. Server perlu menerapkan batas panjang, validasi format, normalisasi yang aman, dan pembuatan ID tepercaya pada boundary aplikasi; request_id bukan bukti identitas maupun otorisasi.

5) Apa yang terjadi bila buffer Fluent Bit mencapai batas 256 MB?

Dalam konfigurasi output pgsql digunakan storage.total_limit_size 256M. Ketika batas antrean output tercapai, Fluent Bit dapat membuang chunk tertua untuk menyediakan ruang bagi chunk baru, sehingga peristiwa lama berisiko hilang meskipun Retry_Limit False diaktifkan. Pengujian ini tidak menguji buffer sampai penuh; kondisi tersebut memerlukan pemantauan kapasitas, alert, dan uji beban terpisah.

6) Bagaimana mendeteksi duplikasi record setelah retry?

Untuk transaksi tertentu, periksa GROUP BY request_id, service, event, timestamp, dan attempt lalu cari COUNT(*) lebih dari satu. Pada Tahap D, empat jenis event yang diharapkan masing-masing ditemukan satu kali, tetapi itu bukan jaminan umum. Sistem produksi lebih baik menambahkan event_id yang unik dan mengaktifkan deduplikasi atau constraint unik yang sesuai di penyimpanan.

7) Kapan PostgreSQL tidak lagi sesuai sebagai backend log utama?

PostgreSQL memadai pada skala lab atau beban terbatas yang membutuhkan SQL dan korelasi sederhana. Jika ingest log sangat tinggi, retensi panjang, volume data besar, pencarian full-text lintas layanan intensif, atau butuh distribusi lintas node dengan latensi rendah, biaya partisi, indeks, dan pemeliharaan dapat menjadi penghambat. Saat itu perlu menilai platform logging khusus berdasarkan beban nyata dan kebutuhan pencarian.

8) Kontrol apa yang harus ditambahkan sebelum arsitektur serupa dipakai pada produksi?

Diperlukan TLS/mTLS antarkomponen, manajemen dan rotasi secret, RBAC, pembatasan jaringan, hardening container, audit akses, enkripsi data, kebijakan retensi dan penghapusan PII, validasi request_id, rate limit, pencatatan event_id, idempotensi dan deduplikasi, backup dan disaster recovery, pemantauan retries_failed/dropped_records/konsumsi buffer, alert, dan pengujian beban serta failure scenario yang terukur.

I. Evidence Minimum

1) Status service

Bukti: Gambar 49 (log-db Exited sedangkan service lain Up) dan Gambar 55 (log-db kembali healthy). Status Up perlu dilengkapi pemeriksaan kesehatan fungsi bisnis.

2) Request ID Baseline dan insiden

Baseline: REQ-b5823bafa0c844a6a82a71114a1ab841. Insiden: REQ-2a8478a049be4888abbb879e680f91c6. Request ID Recovery: REQ-cf02557f843a45eb8fe7ba96f182b97f. Request ID Resilience: REQ-b95f6eed110d405b88f1b49ba8223daa.

3) Query kronologi kedua Request ID

Gambar 30 memperlihatkan empat event INFO pada Baseline, sedangkan Gambar 36 menampilkan tujuh event pada insiden: tiga verification_retry (WARN) dan dependency_timeout (ERROR).

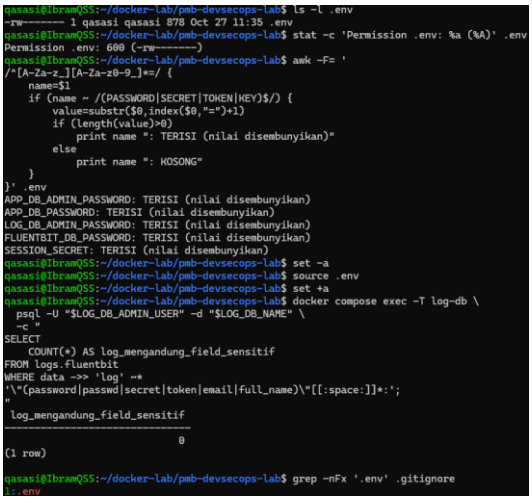
4) Jumlah retry dan kode kegagalan

Transaksi insiden menggunakan 3 percobaan dengan failure_code VERIFY-TIMEOUT (Gambar 34–37). Metrik output Fluent Bit sempat menunjukkan 724 kemudian 1.290 retry, dengan retries_failed dan dropped_records 0 pada snapshot pengujian (Gambar 53 dan 59).

5) Bukti recovery transaksi baru

Gambar 38–42 menunjukkan transaksi Recovery baru berakhir VERIFIED pada percobaan 1, sementara transaksi lama dari Failure Injection tetap FAILED.

6) Hasil pemeriksaan secret



Gambar 72 Cek secret

Perintah pada Gambar 72 digunakan untuk memeriksa keamanan informasi sensitif pada konfigurasi dan centralized logging aplikasi PMB. Hasilnya menunjukkan bahwa file .env hanya dapat diakses oleh pemiliknya dan lima

variabel sensitif telah terisi tanpa menampilkan nilai aslinya. Pemeriksaan pada PostgreSQL tidak menemukan field sensitif sesuai pola pencarian dalam log, sedangkan pemeriksaan .gitignore menunjukkan bahwa file .env telah dikecualikan dari Git. Hasil ini menunjukkan bahwa perlindungan dasar terhadap informasi sensitif telah diterapkan, meskipun belum menjamin seluruh kemungkinan kebocoran data dapat dicegah.

7) Checksum konfigurasi

```
qasasi@IbramQSS:~/docker-lab/pmb-devsecops-lab$ sha256sum compose.yaml compose.override.yaml fluent-bit/fluent-bit.conf
7b17ce965eaa0248c198ca8cb89fc6e9643ed364e0bf535a127d5f2eb4afa411  compose.yaml
391ed3d38c49234986a3d3393431a4d76142209d0bbd51ebbd2d57245e649cd7  compose.override.yaml
1ee2bd0eca7797f4dc3da2c2a9a748ba941911655cf568e22d412e6cc74e536a  fluent-bit/fluent-bit.conf
```

Gambar 73 Cheksum

Perintah pada Gambar 73 digunakan untuk menghasilkan checksum SHA-256 dari file konfigurasi Docker Compose dan Fluent Bit. Hasilnya menunjukkan bahwa ketiga file berhasil diperiksa dan masing-masing memiliki nilai checksum yang dapat digunakan sebagai acuan untuk mendeteksi perubahan konfigurasi. Jika isi file mengalami perubahan, nilai checksum yang dihasilkan akan berbeda.

8) Pernyataan keterbatasan pengujian

Pengujian hanya pada WSL 2 dengan Docker Compose dalam skenario buatan. Keempat event pada satu transaksi Tahap D ditemukan masing-masing sekali, tetapi tidak dilakukan uji buffer penuh 256 MB, beban besar, kehilangan listrik, ketahanan disk, maupun pembuktian exactly-once delivery untuk seluruh event.